

Refinement-based Verification of Cryptographic Protocols with Quantitative Values

Itsaka Rakotonirina
Shielded Technologies
London, United Kingdom
itsaka.rakotonirina@shielded.io

Javier Gomez-Martinez
IMDEA Software Institute
Madrid, Spain
Universidad Politécnica de Madrid
Madrid, Spain
javier.gomez@imdea.org

Aoxuan Li
IMDEA Software Institute
Madrid, Spain
Universidad Politécnica de Madrid
Madrid, Spain
aoxuan.li@imdea.org

Pedro Moreno-Sanchez
IMDEA Software Institute
Madrid, Spain
MPI-SP
Bochum, Germany
pedro.moreno@imdea.org

Clara Schneidewind[✉]
MPI-SP
Bochum, Germany
clara.schneidewind@mpi-sp.org

Abstract

Cryptographic protocols are notoriously hard to design and analyse, giving meaning to analysers such as Tamarin or ProVerif conducting automatic verification in a symbolic model of cryptography. While those tools handle many real-world protocols, they lack support when security relies on quantitative constraints. Examples include time-sensitive protocols, that are crucial for achieving fairness in the presence of a dishonest majority, e.g., to realise fair exchanges.

We present a technique for verifying cryptographic protocols involving critical quantitative features such as timeouts or timelocks. It builds upon existing qualitative analysers as black boxes, to verify a quantitative model through Counter-Example-Guided Abstraction Refinement (CEGAR). It first generates a sound approximated model for the symbolic analyser, checks the potential attack trace it produces, and refines the approximated model based on false alarms. We demonstrate the feasibility of this approach through a fully automated tool using the Tamarin prover as a backend.

Using our tool, we analyse multiple protocols achieving fairness through timed cryptography, e.g., coin flipping or fair signing. Further, we study atomic swap protocols, which enable the exchange of cryptocurrencies among mutually distrusting users. Even though these protocols have been extensively studied in the literature and deployed, our tool identified multiple subtle attack vectors that undermine their security and enable attackers to steal funds.

CCS Concepts

• Security and privacy → Logic and verification; Formal security models.

Keywords

Security Protocols; Formal Methods; Symbolic Models; Real time



This work is licensed under a Creative Commons Attribution 4.0 International License.
CCS '26, The Hague, Netherlands
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2871-6/2026/11
<https://doi.org/10.1145/3830454.3846732>

ACM Reference Format:

Itsaka Rakotonirina, Javier Gomez-Martinez, Aoxuan Li, Pedro Moreno-Sanchez, and Clara Schneidewind. 2026. Refinement-based Verification of Cryptographic Protocols with Quantitative Values. In *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security (CCS '26)*, November 15–19, 2026, The Hague, Netherlands. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3830454.3846732>

1 Introduction

Multi-party protocols are distributed programs executed by mutually untrusted parties, with the underlying goal of achieving a common task, e.g., jointly draw a random string, execute an auction or exchange digital assets across different blockchains. In practice, some parties may have an incentive to deviate from the protocol's specification, or simply abort the computation if the potential output is not of their convenience. To exemplify the issue, consider the scenario where two mutually distrusting parties (A and B) want to agree on a common random string. The straw-man solution is to have A sample a random string x locally and send a commitment of x to B . Next, B samples a random string y and sends it to A , who in turn, reveals the previously committed message and the jointly computed string is set to $z = x \oplus y$. Intuitively, y cannot depend on x (since x is hidden by the commitment), and x cannot be changed after y is sent (since x is previously committed).

This intuitive analysis misses the crucial point that A learns the output of the protocol before B , giving A the upper hand on stalling the protocol if the output is not favourable (e.g., z is even and A prefers odd strings as part of an application-dependent bet with B). In fact, the protocol does not achieve *fairness*, a cryptographic notion that guarantees that either all parties obtain the output of the protocol, or none of them does. Compellingly, it is *provably* known to be impossible to achieve fairness in the presence of a dishonest majority in classic models of asynchronous communication [20], and thus, the design of fair protocols is a notoriously hard task.

Fairness Via Time Delays. Interestingly, a recent work [13] has established that *delay functions*, a class of functions that are assumed not to be computable in less than a chosen amount of time, are

necessary to circumvent the impossibility result of [20] and achieve fairness in the presence of a dishonest majority. Going back to our problem of string sampling, building upon a delay function f , we can construct a fair string sampling protocol as follows: A and B broadcast their random strings x and y within a strict time window, and the output is set to $f(h(x \parallel y))$ where h is a hash function. Neither A nor B can bias the output of the protocol since the input of f is computed as a hash value and both parties learn the output of the protocol after the delay imposed by f .

Many of these so-called *timed cryptographic primitives* have been proposed in the literature. In the spirit of delay functions, they are assumed not to be computable in less than a chosen amount of time [38]. These primitives have been used to provide fairness in the dishonest majority setting for a variety of applications such as sealed-bid auctions [12], fair contract signing [12], randomness beacon [11], or blockchain-based atomic swaps [40].

Symbolic Analysis of Timed Delay Protocols. To alleviate the burden of auditing complex protocols, common solutions employ formal methods to specify the protocol's behaviour rigorously and explore all possible scenarios. One successful research direction is *symbolic models*, in the vein of the seminal work of Dolev and Yao [24]. This refers to a range of techniques modelling a powerful adversary having complete read-write control over communications, but where cryptography is idealised. This leads to protocols being modelled by means of logics or similar calculi, facilitating the automation of their analysis. Recent achievements of state-of-the-art tools include, for example, the analyses of [7, 9, 23, 30].

However, the abstract nature of symbolic models makes them factor away, by design, low-level technicalities. While this still results in realistic models in most cases, this also hinders precision when such low-level parameters are at the core of the protocol's security. In particular, the aforementioned notions of fairness, enforced through carefully engineered arithmetic over quantitative measurements, have notoriously scarce support in the literature. Despite a few attempts to study this problem [5, 37], they were theory-focused and did not lead to fully fledged automated tools.

Contributions

Analysis for Quantitative Protocols. We describe a generic pipeline extending any (black-box access to a) regular symbolic tool, to support quantitative notions. Our methodology is implemented at:

<https://gitlab.com/irakoton/tidy-automation>

- (1) We first formalise the quantitative protocol in a language close to the Sapic tool [18], a powerful language front-end to most modern symbolic analysers.
- (2) The quantitative protocol is first abstracted as a regular one, by erasing quantitative notions from its model. This abstraction is sound for the analysis of safety properties, in that it over-approximates the set of the protocol's executions.
- (3) The verification is discharged to a back-end symbolic analyser, here Tamarin [6]. Successful proofs carry over quantitatively by soundness. Security violations follow a CEGAR approach (*Counter-Example-Guided Abstraction Refinement*), by verifying the validity of the attack against the erased quantitative constraints, using an SMT solver, and excluding invalid attacks from subsequent iterations.

Due to its straightforward flow, this approach should be seen as a way to flag potential quantitative attack vectors, to then automatically sort out genuine attacks from innocuous behaviours. We also highlight that, despite our inspiration from partial, non automated ideas mentioned in early time-based symbolic models [5], the formalisation of our approach and its automation (i.e., everything after Section 3 in our paper) constitute novel contributions.

Case Studies. We analysed multiple protocols achieving fairness built on time delays. First, we considered protocols relying on timed cryptography, including a standard string sampling protocol based on time commitments, randomness beacon protocols and sealed-bid auctions. Our tool provided, for the first time, a fully automated analysis of such quantitative variants of fairness properties. Second, we analyse the more complex case of *atomic swaps*, used in practice to enable the exchange of assets among mutually distrusting users, across different cryptocurrencies. We modelled a version of the protocol from the description of [40], and our tool unveiled two attack vectors during the setup phase that undermine the fairness of the protocol, enabling attackers to steal funds. Further manual inspection concluded that specific checks in the setup phase may be security critical in some contexts, contrasting with the sometimes imprecise description of this phase in the literature [19, 32, 40], which leaves relevant checks underspecified. As our young prototype is not mature enough yet to handle industrial-scale verifications, we bring to attention that these attack vectors have already been found on a model that is significantly less intricate than swaps in practice (no parallel sessions nor cross-chain interactions). These early findings recall the need for a particular attention to details in the security and implementation of this class of protocols.

2 Motivating Example: Delay-Based Sampling

We outline how we handle a protocol, as in the introduction, of two mutually distrustful parties A and B attempting to sample a common string by relying on *Verifiable Delay Functions* (VDF) [11]. VDF are, in short, deterministic functions whose outcome requires a significant, non-parallelisable time Δ to compute, but whose result can be verified easily (i.e., without re-computing the function itself).

2.1 Symbolic Modelling

In our framework derived from the applied pi calculus [1], cryptographic primitives are commonly modelled by *function symbols*:

vdf/1 verify/2 ok/0 h/1.

The vdf/1 symbol indicates an operation of arity 1; in our context, $\text{vdf}(x)$ represents the output of the delay function on input x . On the contrary, $\text{verify}(v, x)$ models the verification of said VDF, which is characterised by the following algebraic equation:

$$\text{verify}(\text{vdf}(x), x) = \text{ok}.$$

This equation states that verifying a VDF against its input yields a successful result, while the absence of other equations implicitly expresses that all other cases fail. Pushing this idea further, having a symbol $h/1$ characterised by no equation at all models that h is treated as a form of random oracle. Concatenation of bitstrings is also standardly modelled by pairs $\langle x, y \rangle$ and projections $\text{proj}_i(p)$, $i \in \{0, 1\}$, with the two equations $\text{proj}_i(\langle x_0, x_1 \rangle) = x_i$. So far, these

symbols bear no form of execution time. Putting that aside for now, the role of A in the protocol can then be described by the *process*:

$$A \triangleq \text{Start} :: \text{new } r_A :: \text{out}(r_A) ; \text{in}(r_B) ; \\ \text{Challenge}(\text{vdf}(\text{h}(\langle r_A, r_B \rangle)))$$

In the first line, A generates a nonce r_A and outputs it, then waits for an input r_B from B . Note in particular the difference between “;” (the standard sequential composition of instructions) and “::” modelling *atomic* composition of two instructions, i.e., two instructions performed as one block, at the same timestamp [37]. The *events* **Start** and **Challenge**(v) are meta instructions (*events*) used to formalise security properties. They identify, here, the moment A reveals their contribution, and the moment B is challenged to guess the value of v . The fact that this challenge must happen in a Δ window can typically be expressed by the (quantitative) *formula* φ_Δ :

$$\varphi_\Delta \triangleq \forall v, i, j. \text{Start}@i \wedge \text{Challenge}(v)@j \implies j < i + \Delta.$$

This simply reads as “*the challenge, if any, must occur at most Δ units of time after the Start event*”. Under this assumption, the unpredictability of the result would therefore be stated as follows: “*for any behaviour of B , B does not learn the value of v before Challenge(v) occurs*”. In our formalism, this is $\varphi \triangleq (\varphi_\Delta \implies \psi)$, where:

$$\psi \triangleq \forall v, i, j. \text{Challenge}(v)@i \wedge K(v)@j \implies j > i$$

Note in particular that $K(v)@j$ (often read as “*Attacker*”) means that a Dolev-Yao *adversary*, namely an entity controlling the inputs and outputs received by, and sent from, A , constructs the value of v at time j . Here the adversary therefore models the party B itself.

The last pending modelling point is to constrain the adversary B not to compute $\text{vdf}(x)$ in time less than Δ . We do so by declaring the vdf symbol as *private* (literally: the adversary cannot compute it), while giving access to a time-constrained oracle. The said oracle is modelled by the following process:

$$O = \text{in}(x) :: @t_0 ; \text{out}(\text{vdf}(x)) :: @t_1 :: \text{when } t_1 \geq t_0 + \Delta$$

This process receives an input x chosen by B at time t_0 , and reveals $\text{vdf}(x)$ at a later time $t_1 \geq t_0 + \Delta$, as captured by this “when” syntax holding arithmetic statements. This process may all in all be used by B to compute one VDF in time Δ . Our formalisation of unpredictability is then that φ holds for any potential execution of the process $P = A \mid !O$ (“*process A put in parallel to arbitrarily many parallel copies of O* ”), within an implicit adversarial environment.

2.2 Verification

Our approach is to front-end symbolic tools to discharge them from any quantitative considerations, using a refinement-based approach à la CEGAR. The pipeline is layered as follows.

Abstraction. Given a concrete verification instance v_c (here: the process P and the formula φ), an *abstraction* α maps v_c to an abstract instance v_a fitting the scope of a regular symbolic analyser. In the example above, α performs the following transformations.

- (1) All blocks of atomic instructions $\beta = \text{instr}_1 :: \dots :: \text{instr}_n$ in P , at position p , turn into $\text{in}(\text{numeric}(t_p)) :: \bar{\beta} :: \text{State}_p(\bar{x})$, where t_p is a fresh variable representing the current time, $\text{numeric}/1$ is a symbol indicating a placeholder for numeric values, and $\text{State}_p(\bar{x})$ is an event indicating the position p in the protocol’s code and the variables $\bar{x}$ in scope (t_p included).

Finally, $\bar{\beta}$ is obtained by removing any timestamping instruction $@s$ from β (and substituting s by $\text{numeric}(t_p)$ in the continuation), as well as replacing each event $\text{Ev}(\bar{y})$ by a variant $\text{Ev}_a(\bar{y}, \text{numeric}(t_p))$ carrying the abstract timestamp.

- (2) The occurrence of when c in process O is replaced by a plain event $\text{SMT}(\bar{c})$ with $\bar{c}$ an uninterpreted representation of c .
- (3) The arithmetic subformula $j < i + \Delta$ of φ is treated as the tautology $\top$, making φ_Δ effectively vacuous. Doing so over-approximates the formula, i.e., more attacks will be found, making the approximation sound.

In our example, we get the abstract process in Figure 1. The process $P = A \mid !O$ has seven block positions, named for legibility: ε (the root), s, i, c (the three blocks of A), $!$ (the replication) and o_1, o_2 (the two blocks of O). Quantitative features have thus been removed: timestamps $@t_i$ are substituted by adversarial inputs $\text{numeric}(t_{o_i})$, the delay Δ has become an uninterpreted public name $\bar{\Delta}$, and the guard “when” has become the event $\text{SMT}(\cdot)$, always executable. On the formula side, the abstraction is $\varphi_a \triangleq (\varphi_\Delta^a \implies \psi_a)$ with

$$\varphi_\Delta^a = \forall v, i, \bar{i}, j, \bar{j}. \\ \text{Start}_a(\text{numeric}(\bar{i}))@i \wedge \text{Challenge}_a(v, \text{numeric}(\bar{j}))@j \\ \implies \text{numeric}(\bar{j}) < \text{numeric}(\bar{i}) + \bar{\Delta} \\ \psi_a = \forall v, i, \bar{i}, j. \\ \text{Challenge}_a(v, \text{numeric}(\bar{i}))@i \wedge K(v)@j \implies j > i.$$

Each quantified timepoint i is duplicated into a plaintext copy $\bar{i}$, which the events of $\alpha(P)$ now carry as an extra argument: this is how a formula gets a handle on an erased numeric value, as timepoints $@i$ and messages have different statuses in standard tools like Tamarin. The inequality of φ_Δ^a is the one the back-end cannot interpret, so although it is kept for attack-verification purposes, it will be overwritten as $\top$ when sent to the back-end. In contrast, the $j > i$ of ψ_a is a plain temporal ordering, supported natively.

Oracle. The verification is then discharged to the *oracle* O , representing the back-end analyser. Our only assumption is that, upon successful verification, the oracle returns either a sound security proof, or an explicit (abstract) counterexample execution of P .

In the unpredictability example, the oracle unsurprisingly finds a straightforward attack. Since all time constraints (the “when” check and the hypothesis φ_Δ) have been filtered out, the following sequence of actions T_a of $\alpha(P)$ violates φ in the abstract model. We assume fresh public names r_B, t_p , for each position p , and write $m \triangleq \text{h}(\langle r_A, r_B \rangle)$.

- (ε) The run starts, triggering $\text{State}_\varepsilon(t_\varepsilon)$. Recall that, in this qualitative model, t_ε is simply an uninterpreted name.
- (s) A executes the **Start** block, outputting r_A , yielding events $\text{Start}_a(\text{numeric}(t_s))$ and $\text{State}_s(t_s, t_s, r_A)$.
- ($!$) Triggers $\text{State}_!(t_\varepsilon, t_!)$, and spawns a copy of O .
- (o_1) B sends, to that copy of O , the input $x = m$, built using the previous output of r_A .
- (o_2) The copy of O reveals $v = \text{vdf}(m)$, *without delay*, since the guard has become the non-blocking, and thus vacuous, event $\text{SMT}(\bar{c})$ with $\bar{c} = \text{numeric}(t_{o_2}) \geq \text{numeric}(t_{o_1}) + \bar{\Delta}$.
- (k) B grabs the value v , which the trace records as the event $K(v)$. Unlike the other steps, this one is an action of the adversary rather than a block of $\alpha(P)$, and carries no **State** event.
- (i) B sends r_B to A , triggering $\text{State}_i(t_\varepsilon, t_s, r_A, t_i, r_B)$.

$$\begin{aligned}
\alpha(P) &= \text{in}(\text{numeric}(t_\varepsilon)) :: \text{State}_\varepsilon(t_\varepsilon) :: (A_a \mid O_a) \\
A_a &= \text{in}(\text{numeric}(t_s)) :: \text{Start}_a(\text{numeric}(t_s)) :: \text{new } r_A :: \text{out}(r_A) :: \text{State}_s(t_s, t_s, r_A); \\
&\quad \text{in}(\text{numeric}(t_i)) :: \text{in}(r_B) :: \text{State}_i(t_s, t_s, r_A, t_i, r_B); \\
&\quad \text{in}(\text{numeric}(t_c)) :: \text{Challenge}_a(\text{vdf}(\text{h}(\langle r_A, r_B \rangle)), \text{numeric}(t_c)) :: \text{State}_c(t_s, t_s, r_A, t_i, r_B, t_c) \\
O_a &= \text{in}(\text{numeric}(t_l)) :: \text{State}_l(t_s, t_l) :: ! (\text{in}(\text{numeric}(t_{o_1})) :: \text{in}(x) :: \text{State}_{o_1}(t_s, t_l, t_{o_1}, x); \\
&\quad \text{in}(\text{numeric}(t_{o_2})) :: \text{out}(\text{vdf}(x)) :: \text{SMT}(\text{numeric}(t_{o_2}) \geq \text{numeric}(t_{o_1}) + \Delta) :: \text{State}_{o_2}(t_s, t_l, t_{o_1}, x, t_{o_2}))
\end{aligned}$$

Figure 1: Process abstraction $\alpha(P)$ of $P = A \mid !O$

$$\begin{aligned}
\text{Excl}(T_a) &= \forall y_A, y_B, z_\varepsilon, z_s, z_i, z_c, z_l, z_{o_1}, z_{o_2}, j_\varepsilon, j_s, j_i, j_c, j_l, j_{o_1}, j_{o_2}. \\
&\quad \text{State}_\varepsilon(z_\varepsilon)@j_\varepsilon \wedge \text{State}_s(z_s, y_A)@j_s \wedge \text{State}_i(z_i, y_A, z_i, y_B)@j_i \\
&\quad \wedge \text{State}_c(z_\varepsilon, z_s, y_A, z_i, y_B, z_c)@j_c \wedge \text{State}_l(z_\varepsilon, z_l)@j_l \\
&\quad \wedge \text{State}_{o_1}(z_\varepsilon, z_l, z_{o_1}, \text{h}(\langle y_A, y_B \rangle))@j_{o_1} \wedge \text{State}_{o_2}(z_\varepsilon, z_l, z_{o_1}, \text{h}(\langle y_A, y_B \rangle), z_{o_2})@j_{o_2} \\
&\implies j_\varepsilon \geq j_s \vee j_s \geq j_i \vee j_i \geq j_c \vee j_c \geq j_l \vee j_l \geq j_{o_1} \vee j_{o_1} \geq j_{o_2} \vee j_s \geq j_{o_1}
\end{aligned}$$

Figure 2: The restriction $\text{Excl}(T_a)$ excluding the spurious attack T_a from the next iterations

- (c) A triggers the two events $\text{Challenge}_a(v, \text{numeric}(t_c))$, and $\text{State}_c(t_s, t_s, r_A, t_i, r_B, t_c)$. This violates the abstraction of φ : the only instance of ψ_a pairs this Challenge_a event with the $K(v)$ event of Step (k), which occurred earlier.

Concretisation. If the oracle result is negative as above, the attack candidate T_a may or may not be a violation of the concrete instance v_c . Checking this is the role of the *concretisation* γ . It will compute an SMT query Γ corresponding to the validity of the attack. This is done over one unknown $\tau_p \in \mathbb{R}_+^*$ per placeholder t_p , standing for the real time at which the block p was executed.

- (1) First, Γ_{ord} are the *ordering constraints* of T_a , namely, the constraints $\tau_p < \tau_q$ expressing that two steps p, q must occur one after another. This can be either due to their positions in the process, or a data dependency (the adversary using a message previously revealed). Both kinds of edges are collected in a dependency graph $G(T_a)$, here, using the labels from the attack description:

$$\varepsilon \rightarrow s \rightarrow i \rightarrow c \quad \varepsilon \rightarrow ! \rightarrow o_1 \rightarrow o_2 \rightarrow k \quad s \rightarrow o_1$$

$$\begin{aligned}
\text{Hence: } \Gamma_{\text{ord}}(T_a) &= \tau_\varepsilon < \tau_s \wedge \tau_s < \tau_i \wedge \tau_i < \tau_c \\
&\quad \wedge \tau_\varepsilon < \tau_l \wedge \tau_l < \tau_{o_1} \wedge \tau_{o_1} < \tau_{o_2} \\
&\quad \wedge \tau_{o_2} < \tau_k \wedge \tau_s < \tau_{o_1}.
\end{aligned}$$

- (2) Second, Γ_{when} is the conjunction of all c such that $\text{SMT}(\bar{c})$ appears in T_a . Here the single such event, of Step (o_2), is read back as an actual arithmetic statement, giving:

$$\Gamma_{\text{when}}(T_a) = \tau_{o_2} \geq \tau_{o_1} + \Delta.$$

- (3) Third, convert φ_a into a boolean formula Γ_{form} as well, by interpreting $\forall$ quantifiers as conjunctions and $\exists$ as disjunctions over the events actually occurring in T_a , and by replacing every placeholder $\text{numeric}(t_p)$ by its unknown τ_p introduced earlier. The hypothesis φ_a^a has exactly one instance in T_a , pairing the **Start** event of Step (s) with the **Challenge** event of Step (c), and thus contributes $\tau_c < \tau_s + \Delta$. The conclusion ψ_a , on the other hand, requires the adversary to derive v

strictly after the challenge, which contributes $\tau_k > \tau_c$. Altogether:

$$\Gamma_{\text{form}}(T_a, \varphi_a) = (\tau_c < \tau_s + \Delta) \implies (\tau_k > \tau_c).$$

The attack is genuine if φ is *violated*, so the query aggregates as:

$$\Gamma = \Gamma_{\text{ord}} \wedge \Gamma_{\text{when}} \wedge \neg \Gamma_{\text{form}}.$$

It is unsatisfiable (i.e., the attack is spurious), with the following UNSAT core:

$$\begin{array}{lll}
\tau_s < \tau_{o_1} & \tau_{o_1} + \Delta \leq \tau_{o_2} & \tau_{o_2} < \tau_k \\
\tau_k \leq \tau_c & \tau_c < \tau_s + \Delta &
\end{array}$$

the last two conjuncts coming from $\neg \Gamma_{\text{form}}$. Indeed, chaining them gives $\tau_s < \tau_{o_1} \leq \tau_{o_2} - \Delta < \tau_k - \Delta \leq \tau_c - \Delta < \tau_s$, a contradiction. Rephrasing, the adversary would have to obtain the value of the delay function strictly less than Δ units of time after choosing its input, which is precisely what the process O forbids.

Refinement. Upon invalidating the attack T_a like above, the abstract instance v_a must be refined to avoid stumbling on T_a again in subsequent iterations. This is the role of the *refinement* ρ , which constructs a formula $\text{Excl}(T_a)$ characterising exactly the execution flow of T_a , using the events State_p added during the abstraction, and the ordering constraints Γ_{ord} computed previously.

Concretely, all names that T_a generated at run time (the fresh names r_A, r_B , time placeholders $t_p \dots$) are turned into universally quantified variables (resp. y_A, y_B and z_p), so that the refinement excludes not only T_a but every variant of it obtained by alpha renaming of fresh names. The premise then replays the **State** events of T_a in that renamed form, and the conclusion states that at least one of the orderings of $G(T_a)$ must be broken, as given in Figure 2. The analysis then starts over from the oracle call O , but proving $\text{Excl}(T_a) \implies \varphi$ instead of φ ; in practice this is passed to Tamarin as a *restriction*. On this example the loop stops there: the second call to the oracle returns a proof, and unpredictability is therefore established for P . We include the automated analysis of this example in our case studies, see Section 5, Figure 6.

3 Model

We now formally introduce our symbolic model, a stateful variant of the applied π -calculus. The model is mostly inspired by [37].

3.1 Cryptographic Primitives and Messages

In symbolic models, cryptographic messages are usually modelled by so-called *terms*. They first involve *atomic values*:

$$A = N \uplus X \quad \text{with} \quad N = N_{\text{pub}} \uplus N_{\text{priv}}.$$

The set N of *names* contains *public names* (N_{pub}), which capture for example identities or public constants. *Private names* (N_{priv}) model data unknown to the adversary such as long-term cryptographic keys. The set X then contains *variables*, in the usual sense. In the quantitative setting, we consider a minimal, implicit type system, in the sense that some objects represent numeric values, and others, plain cryptographic material. In practice, we assume $\mathbb{R} \subset N_{\text{pub}}$ and let $X^N \subset X$: that is, real numbers are treated as valid public values, and we consider a distinguished set X^N of *numeric variables*. We add the usual standard assumptions that there are infinitely many names and variables of both types. We write $\text{vars}(u)$ the set of variables appearing in an arbitrary object u . Next, we define *signatures*, which refer to finite sets

$$\mathcal{F} = \{f/n, g/m, h/p, \dots\} = \mathcal{F}_{\text{pub}} \uplus \mathcal{F}_{\text{priv}}$$

modelling cryptographic primitives, like `vdf/1` and `verify/2` in Section 2. We recall the distinction of `vdf` being *private* ($\text{vdf} \in \mathcal{F}_{\text{priv}}$) expresses a constraint that the adversary is not able *a priori* to compute the output of `vdf`. Put together, we get:

Definition 3.1 (Term). A *term* is an atomic value, or a function symbol or arithmetic operator ($+$, $\times$, $-$, $\dots$), applied to other terms while respecting arities. We write $\mathcal{T}(S)$ the set of terms built from $S \subseteq A \cup \mathcal{F} \cup O$, for a given set of arithmetic operators O . We call a term $u \in \mathcal{T}(S)$: *ground* if $S \cap X = \emptyset$; *public* if $S \cap N_{\text{priv}} = \emptyset$ and $S \cap \mathcal{F}_{\text{priv}} = \emptyset$; *numeric* if $S \subseteq \mathbb{R} \cup X^N \cup O$.

Finally, standard *substitutions* σ are defined as usual:

$$\sigma = \{x_1 \mapsto u_1, \dots, x_n \mapsto u_n\}$$

is a mapping where $\text{dom}(\sigma) = \{x_1, \dots, x_n\}$ are pairwise distinct variables (*domain* of σ), and $u_1, \dots, u_n$ are terms. We say that σ is then *well-typed* if, for all i such that $x_i \in X^N$, u_i is a numeric term. The application of σ to a term t , written $t\sigma$, is the result of replacing all occurrences of x_i in t by $u_i = x_i\sigma$. Also, more specifically, we say that a term C is a *context* if, writing $\text{vars}(C) = \{x_1, \dots, x_n\}$, each x_i appears exactly once in C . In this case, we write:

$$C[u_1, \dots, u_n] \quad \text{instead of} \quad C\{x_1 \mapsto u_1, \dots, x_n \mapsto u_n\}.$$

We always implicitly require that the context application is well-typed, that is, that $\{x_1 \mapsto u_1, \dots, x_n \mapsto u_n\}$ is well-typed.

Terms have an operational behaviour that is classically modelled by an *equational theory* E . This typically represents properties such as the verification of VDFs in Section 2. Generically speaking:

Definition 3.2 (Equational theory). An *equational theory* E is a set of pairs of terms without arithmetic operators. It induces a relation $=_E$ on terms, called *equality modulo* E , that is the smallest equivalence relation containing E and such that $u =_E v$ implies that for all contexts C and well-typed substitutions σ , $C[u\sigma] =_E C[v\sigma]$.

We then simply call a *term algebra* a pair

$$\mathbb{A} = (\mathcal{F}, E)$$

with E an equational theory for terms over the signature $\mathcal{F}$. Unlike in most papers in the field, we do not assume an implicit, fixed term algebra throughout our technical contributions. The term algebra will indeed not always be clear from context, and we will therefore parametrise all associated notions by it.

3.2 Protocols in an Adversarial Environment

Syntax. We model protocols as *processes* of a quantitative and stateful variant of the applied π -calculus [1, 37].

Definition 3.3 (Process). Processes P are defined by the following grammar, where $x, k \in X$, $t \in X^N$, and $u, u_1, \dots, u_n$ are terms:

$$\begin{array}{lll} P ::= & 0 & S \quad P \mid P \quad !P \\ S ::= & I :: S & \text{either } S \text{ or } S \quad \text{skip} ; P \\ I ::= & \text{in}(u) & \text{push } u \quad \text{Ev}(u_1, \dots, u_n) \\ & \text{out}(u) & \text{pull } u \quad @t \\ & \text{new } k & \text{unfounded } u \quad \text{when } c \end{array}$$

We comment the syntax below. Processes consist of instructions (token I of the grammar), that are composed through structural operators (tokens P and S). This includes $P \mid Q$ which executes P and Q concurrently, $!P$ which executes arbitrarily many copies of P in parallel, and either P or Q which executes only one of two processes. The latter is called (*external*) *non-deterministic choice* in the literature [4]. Although it is usually written $P + Q$, we preferred a keyword-based syntax for readability.

Instructions are composed by either a symbol “;” (preceded by a skip), which refers to the usual sequential composition, or by an *atomic composition* “::” [37]. Intuitively, in an atomic sequence of instructions such as:

$$i_1 :: i_2 :: \text{skip} ; i_3 :: i_4 :: i_5 :: \text{skip} ; 0$$

the first instructions i_1 and i_2 are executed simultaneously, followed by the block i_3 – i_5 after an arbitrary (non-null) amount of time has passed. For concision, we will usually write such a sequence:

$$i_1 :: i_2 ; i_3 :: i_4 :: i_5.$$

Regarding the instructions themselves, `new` k generates a fresh name, i.e., a value unknown to the adversary. Events are used to refer to specific parts of a protocol execution when formalising security properties, recall **Start** and **Challenge** from Section 2. We also recall timestamps `@t`, often atomically composed to a `when` c to measure time and impose arithmetic conditions on it.

Processes also refer to a global state, i.e., a multiset θ of terms shared by all parallel processes. Terms can be added to θ using `push` u , and pulled (with pattern matching) using `pull` u . In short, `pull` u checks whether there exists a substitution σ such that $u\sigma \in \theta$, and removes one such occurrence of $u\sigma$ from θ ; it *blocks* when no such substitution exists. Dually, `unfounded` u can be executed if, and only if, there is no such substitution, and blocks otherwise.

Finally, inputs and outputs handle communications in an adversarial environment. An output `out`(u) reveals u to the adversary, while an input `in`(u) receives an arbitrary term x from them, computed from previous outputs, and matched against the pattern u .

Atomic semantics (over single processes):

$(\text{out}(v) :: S, \Phi, \mathbf{K}, \theta, t) \rightarrow_{\text{at}} (S, \Phi \cup \{\!\! v \!\!\}, \mathbf{K}, \theta, t)$		(OUT)
$(\text{in}(u) :: S, \Phi, \mathbf{K} \cup \{\!\! v \!\!\}, \theta, t) \rightarrow_{\text{at}} (S\sigma, \Phi, \mathbf{K}, \theta, t)$	for some substitution σ such that $u\sigma =_E v$	(IN)
$(\text{new } k :: S, \Phi, \mathbf{K}, \theta, t) \rightarrow_{\text{at}} (S\{k \mapsto k'\}, \Phi, \mathbf{K}, \theta, t)$	if $k' \in \mathcal{N}_{\text{priv}}$ fresh	(NEW)
$(\text{Ev}(\vec{u}) :: S, \Phi, \mathbf{K}, \theta, t) \xrightarrow{\text{Ev}(\vec{u})}_{\text{at}} (S, \Phi, \mathbf{K}, \theta, t)$		(EVENT)
$(\text{push } u :: S, \Phi, \mathbf{K}, \theta, t) \rightarrow_{\text{at}} (S, \Phi, \mathbf{K}, \theta \cup \{\!\! u \!\!\}, t)$		(PUSH)
$(\text{pull } u :: S, \Phi, \mathbf{K}, \theta \cup \{\!\! v \!\!\}, t) \rightarrow_{\text{at}} (S\sigma, \Phi, \mathbf{K}, \theta, t)$	for some substitution σ such that $u\sigma =_E v$	(PULL)
$(\text{unfound } u :: S, \Phi, \mathbf{K}, \theta, t) \rightarrow_{\text{at}} (S, \Phi, \mathbf{K}, \theta, t)$	if for all substitutions σ , all $v \in \theta$, $u\sigma \neq_E v$	(UNFOUNDED)
$(\text{@}_{t_0} :: S, \Phi, \mathbf{K}, \theta, t) \rightarrow_{\text{at}} (S\{t_0 \mapsto t\}, \Phi, \mathbf{K}, \theta, t)$		(STAMP)
$(\text{when } c :: S, \Phi, \mathbf{K}, \theta, t) \rightarrow_{\text{at}} (S, \Phi, \mathbf{K}, \theta, t)$	if c is interpretable modulo $\mathbb{A}$ as a true arithmetic statement	(WHEN)
$(\text{either } S_0 \text{ or } S_1, \Phi, \mathbf{K}, \theta, t) \rightarrow_{\text{at}} (S_i, \Phi, \mathbf{K}, \theta, t)$	if $i \in \{0, 1\}$	(CHOICE)

Sequential semantics:

$(\{\!\! P \mid Q \!\!\}, \Phi, \mathbf{K}, \theta, t) \rightarrow_{\text{seq}} (\{\!\! P, Q \!\!\}, \Phi, \mathbf{K}, \theta, t)$		(PAR)
$(\{\!\! P \!\!\}, \Phi, \mathbf{K}, \theta, t) \rightarrow_{\text{seq}} (\{\!\! P, P \!\!\}, \Phi, \mathbf{K}, \theta, t)$		(REPL)
$(\{\!\! \text{skip} ; P \!\!\}, \Phi, \mathbf{K}, \theta, t) \rightarrow_{\text{seq}} (\{\!\! P \!\!\}, \Phi, \mathbf{K}, \theta, t)$		(SKIP)
$(\{\!\! P \!\!\}, \Phi, \mathbf{K}, \theta, t) \xrightarrow{\alpha, \vec{w}}_{\text{seq}} (\mathcal{P}, \Phi'', \mathbf{K}'', \theta'', t)$	if $(P, \Phi, \mathbf{K}, \theta, t) \xrightarrow{\alpha}_{\text{at}} (P', \Phi', \mathbf{K}', \theta', t)$ and $(\{\!\! P' \!\!\}, \Phi', \mathbf{K}', \theta', t) \xrightarrow{\vec{w}}_{\text{seq}} (\mathcal{P}, \Phi'', \mathbf{K}'', \theta'', t)$	(ATOMIC-STEP)

Operational semantics:

$(\{\!\! P \mid Q \!\!\}, \Phi, \mathbf{K}, \theta, t) \xrightarrow{\vec{w}} (\mathcal{P} \cup Q, \Phi', \mathbf{K}', \theta', t + \delta)$	if $\delta > 0$ and $(\{\!\! P \!\!\}, \Phi, \mathbf{K}, \theta, t + \delta) \xrightarrow{\vec{w}}_{\text{seq}} (\mathcal{P}, \Phi', \mathbf{K}', \theta', t + \delta)$	(LIFT)
$(\mathcal{P}, \Phi, \mathbf{K}, \theta, t) \rightarrow (\mathcal{P}, \Phi \cup \{\!\! \xi\sigma \!\!\}, \mathbf{K}, \theta, t + \delta)$	if $\delta > 0$, ξ public term, $\xi\sigma$ ground, and $\text{img}(\sigma) \subseteq \Phi$	(DEDUCE)
$(\mathcal{P}, \Phi, \mathbf{K}, \theta, t) \xrightarrow{\text{K}(u)} (\mathcal{P}, \Phi, \mathbf{K} \cup \{\!\! u \!\!\}, \theta, t + \delta)$	if $\delta > 0$ and $u \in \Phi$	(ATK)

Figure 3: Operational semantics of the calculus (for a given term algebra $\mathbb{A}$)

Semantics. We now define a labelled transition system describing the execution of processes in an adversarial environment, mostly adapted from [37], and simplified for our more rudimentary class of security properties. The semantics builds upon an extended form of process recording the multiset of subprocesses executed in parallel $\mathcal{P}$, the current adversary's knowledge Φ (as well as the subset $\mathbf{K}$ the attacker is computing for input interference), the global state θ , and the current time t .

Definition 3.4 (Extended process). An *extended process* is a tuple $A = (\mathcal{P}, \Phi, \mathbf{K}, \theta, t)$, with $\mathcal{P}$ a multiset of processes, $\Phi, \mathbf{K}, \theta$ multisets of terms, and $t \in \mathbb{R}^+$. We interpret a plain process P as $(\{\!\!| P |\!\!\}, \emptyset, \emptyset, 0)$.

The operational semantics is then a transition relation $A \xrightarrow{\vec{w}} B$ where A, B are extended processes and $\vec{w}$ is a (possibly-empty) sequence of ground event terms $\text{Ev}(\vec{u})$, including the potential special event $\text{K}(u)$ indicating that the adversary constructs the term u with the intent to send it to the network. The semantics is defined in Figure 3 and introduces two intermediary relations $\rightarrow_{\text{at}}$ (*atomic semantics*) and $\rightarrow_{\text{seq}}$ (*sequential semantics*) to account for the presence of atomic composition in the framework. On the one hand, $\rightarrow_{\text{at}}$ handles a single instruction within an atomic block; on the other hand, $\rightarrow_{\text{seq}}$ chains multiple $\rightarrow_{\text{at}}$ -steps until reaching a skip or a parallel structure. Rule (LIFT) then defines the operational semantics $\rightarrow$ by letting some time $\delta > 0$ pass, and then executing one instance of $\rightarrow_{\text{seq}}$ on a process P in a context of parallel processes Q . Rules (ATK) and (DEDUCE) manage the adversarial interferences.

We now comment more technically on the rules of Figure 3. In the communication rules (OUT) and (IN), outputs build the adversary's knowledge Φ (in combination with Rule (DEDUCE)), in turn

used to compute inputs. Note that, each time it needs to be used in inputs, the knowledge of Φ has to flow to $\mathbf{K}$ through Rule (ATK). This subtlety is simply here to be consistent with the definition of the $\mathbf{K}$ event of the Tamarin back-end, which is defined analogously. Rule (EVENT) triggers an event used to formalise security properties, by highlighting it as a label of the transition. The three rules (PUSH), (PULL) and (UNFOUNDED) then formalise adding to, removing from, or checking the global state θ , respectively. Rule (NEW) generates a fresh private value, and (STAMP) records the current time. Rules (PAR) and (REPL) spawn parallel threads, whereas (CHOICE) atomically branches to either of two processes. Altogether, we get the following notion of execution:

Definition 3.5 (Trace). Let $\mathbb{A}$ be a term algebra, and A_0 an extended process over $\mathbb{A}$. An $\mathbb{A}$ -*trace* of A_0 is a sequence $T : A_0 \xrightarrow{\vec{w}_1} \dots \xrightarrow{\vec{w}_n} A_n$ of transitions as defined in Figure 3 for $\mathbb{A}$.

3.3 Verification Problem

We focus in this paper on *trace properties*, formalised in a first-order logic that is largely standard in the field, albeit for the presence of arithmetic statements.

Definition 3.6 (Formula). The grammar of *formulae* φ is as follows, where $u, v, \vec{u}$ are terms, Ev is an event (or $\mathbf{K}$), $x \in \mathcal{X}$, and $t \in \mathcal{X}^N$:

$$\varphi ::= u = v \quad \varphi \Rightarrow \varphi \quad \forall x. \varphi \quad u > 0 \quad \text{Ev}(\vec{u})@t$$

The logic includes first-order quantification $\forall x$ over terms, or numeric terms if $x \in \mathcal{X}^N$. Also, $u = v$ represents equality modulo theory, and $u > 0$ checks that the numeric term u is positive. The remaining connectives are omitted as they are definable as usual, writing $\perp$ as a trivially false equality, $\neg\varphi$ for $\varphi \Rightarrow \perp$, then $\varphi \vee \psi$

$T \models_{\mathbb{A}} u = v$	iff	$u =_E v$
$T \models_{\mathbb{A}} u > 0$	iff	there exists $p \in \mathbb{R}$ such that $u =_E p$ (interpreting arithmetic operators as real functions) and $p > 0$
$T \models_{\mathbb{A}} \text{Ev}(\vec{u})@t$	iff	there exists $\ell \in T@t$ such that $\ell =_E \text{Ev}(\vec{u})$
$T \models_{\mathbb{A}} \forall x. \varphi$	iff	for all well-typed substitutions σ such that $\text{dom}(\sigma) = \{x\}$, we have $T \models_{\mathbb{A}} \varphi \sigma$
$T \models_{\mathbb{A}} \varphi \Rightarrow \psi$	iff	$T \not\models_{\mathbb{A}} \varphi$ or $T \models_{\mathbb{A}} \psi$

Figure 4: Satisfiability relation for formulae (for a given term algebra $\mathbb{A} = (\mathcal{F}, E)$)

for $\neg\varphi \Rightarrow \psi$, $\varphi \wedge \psi$ for $\neg(\varphi \Rightarrow \neg\psi)$, and $\exists x. \varphi$ for $\neg\forall x. \neg\varphi$. The formula $\text{Ev}(\vec{u})@t$ states that an event occurs at time t . Note that, since terms may contain arithmetic operations $(+, -, \dots)$, the logic can express that some event $\text{Ev}_1@t_1$ occurs before $\text{Ev}_2@t_2$ using a formula $t_1 < t_2$ encoded as $t_2 - t_1 > 0$.

The satisfiability relation for formulae is then formalised in Figure 4, through judgements $T \models_{\mathbb{A}} \varphi$ with T and φ a trace and a formula, respectively, over the term algebra $\mathbb{A}$. Figure 4 uses in particular the following notion of *trace state* $T@t$: it is, intuitively, the set of event labels occurring at time t .

Definition 3.7 (Trace state). Let $T : A_0 \xrightarrow{\vec{w}_1} \dots \xrightarrow{\vec{w}_n} A_n$ be a trace, writing $A_i = (\mathcal{P}_i, \Phi_i, \mathbf{K}_i, \theta_i, t_i)$. If $t \geq t_0$, the *state of T at time t* is defined as $T@t = \{\vec{w}_i\}$ if there is some (necessarily unique) $i \in [1, n]$ such that $t = t_i$, and $T@t = \emptyset$ otherwise.

Despite our logic's minimality, standard features can easily be encoded into this small core. We refer to [37] for details, but one may mention assertions ($\text{assert } \varphi$), that can be lifted to if-then-else conditionals using either-or, let bindings let *pattern* = u in S , or other logical operators $\exists x. \varphi$, $\wedge$, $\vee$, $\neg$ in formulae. The problem we are interested in is then the following. A *verification instance* is

$$v = (\mathbb{A}, P, \varphi)$$

where $\mathbb{A}$ is a term algebra, and P and φ are a process and a formula, respectively, over $\mathbb{A}$.

Verif

Input: A verification instance $v = (\mathbb{A}, P, \varphi)$.

Question: Does it hold that for all $\mathbb{A}$ -traces T of P , $T \models_{\mathbb{A}} \varphi$?

4 Refinement-Based Analysis

In this section, we detail our approach for the Verif problem, formalising the outline previously presented in Section 2.

4.1 Theoretical Interface with Existing Tools

We first present our theoretical framework and the lemmas justifying its correctness. At this point, the presentation is not specific to cryptographic protocols, and is back-end agnostic. It should thus be seen as a declination of CEGAR adapted to our context, and the technical justification of our approach's correctness.

Verification Algorithm. We rely on the following components:

- (1) Two sets V_c and V_a of so-called *concrete* and *abstract instances*, respectively.
- (2) To each $v \in V_c \uplus V_a$, we associate a set $\llbracket v \rrbracket$ called its *semantics*, as well as its *counterexample semantics* $\llbracket v \rrbracket^- \subseteq \llbracket v \rrbracket$.

In practice, concrete instances v_c will be verification instances in the sense of Section 3.3, while abstract ones v_a will be the watered

down version sent to the back-end tool. Concrete and abstract semantics will be the sets of traces of these instances, or only those violating the security property φ in the case of the counterexample semantics. For this reason, we often refer to the elements of $\llbracket v \rrbracket$ as *concrete* or *abstract traces*, depending on the nature of the instance v . Note also that our goal becomes, given $v_c \in V_c$, to prove that $\llbracket v_c \rrbracket^- = \emptyset$. We then operate with the following functions:

- (3) An *oracle* O , mapping abstract instances $v_a \in V_a$ to

$$O(v_a) \in \{\text{accept}\} \cup \{\text{reject}_T \mid T \in \llbracket v_a \rrbracket\}.$$

- (4) An *abstraction* $\alpha : V_c \rightarrow V_a$.
- (5) A *concretisation* $\gamma_{v_c} : \llbracket \alpha(v_c) \rrbracket^- \rightarrow \llbracket v_c \rrbracket \cup \{\perp\}$ which is parametrised by a concrete instance v_c .
- (6) A *refinement* $\varrho_T : V_a \rightarrow V_a$ which is parametrised by an abstract trace T .

Intuitively, α simplifies a concrete instance v_c so that a back-end tool, represented by the oracle O , may verify $\alpha(v_c)$. A call to $O(\alpha(v_c))$ may then either return *accept* (successful proof), or *reject_T* (attack trace T). Successful proofs will be transferable from $\alpha(v_c)$ to v_c , but upon a *reject_T*, the abstract counterexample $T \in \llbracket \alpha(v_c) \rrbracket^-$ has to be checked for validity against the initial instance $\llbracket v_c \rrbracket$. This is the role of the concretisation: $\gamma_{v_c}(T) = \perp$ indicates that T is not a valid concrete attack—in which case the refinement ϱ_T is used to exclude T from the analysis. The whole procedure is formalised in Algorithm 1.

Algorithm 1 Verification procedure

Require: Oracle O , abstraction α , concretisation γ , refinement ϱ

```

1: procedure verify( $v_c$ )
2:   cegar( $v_c, \alpha(v_c)$ )
3: end procedure
4: procedure cegar( $v_c, v_a$ )
5:   if  $O(v_a) = \text{reject}_T$  then
6:     if  $\gamma_{v_c}(T) \neq \perp$  then
7:       return  $\text{reject}_{\gamma_{v_c}(T)}$ 
8:     else
9:       return cegar( $v_c, \varrho_T(v_a)$ )
10:    end if
11:  else
12:    return accept
13:  end if
14: end procedure

```

In real implementations, O , α , γ_{v_c} , ϱ_T may be partial functions, i.e., fail on some inputs and interrupt the whole Algorithm 1. This may come from undecidability reasons, or to enforce requirements

such as Propositions 4.1 to 4.3 below. Since the correctness of Algorithm 1 (Theorem 4.4) is conditional to effective acceptance or rejection, this does not affect the validity of our results.

Correctness of the Algorithm. The correctness of Algorithm 1 relies on the following properties. First of all, the oracle $\mathcal{O}$ should yield accurate results on the abstract side.

PROPOSITION 4.1 (ORACLE CORRECTNESS). *If $v_a \in V_a$:*

- (1) Soundness: *if $\mathcal{O}(v_a) = \text{accept}$ then $\llbracket v_a \rrbracket^- = \emptyset$.*
- (2) Completeness: *if $\mathcal{O}(v_a) = \text{reject}_T$ then $T \in \llbracket v_a \rrbracket^-$.*

This will be in practice a property of the back-end tool, and therefore an assumption on our side. Next comes the dual relation between the abstraction and the concretisation.

PROPOSITION 4.2 (ABSTRACTION CORRECTNESS). *If $v_c \in V_c$:*

- (1) Soundness: *for all $T_c \in \llbracket v_c \rrbracket^-$, there exists $T_a \in \llbracket \alpha(v_c) \rrbracket^-$ such that $\gamma_{v_c}(T_a) = T_c$.*
- (2) Completeness: *for all $T_a \in \llbracket \alpha(v_c) \rrbracket^-$, if $\gamma_{v_c}(T_a) \neq \perp$ then $\gamma_{v_c}(T_a) \in \llbracket v_c \rrbracket^-$.*

Soundness states that all concrete counterexamples have an abstract counterpart, which justifies that successful abstract proofs carry over to their corresponding concrete instance. Dually, completeness states that abstract attacks that pass the validity check are also attacks in the concrete instance. The final requirement below is then about the refinement, namely that it excludes invalid traces.

PROPOSITION 4.3 (REFINEMENT CORRECTNESS). *If $v_c \in V_c, v_a \in V_a$ with $\llbracket v_a \rrbracket^- \subseteq \llbracket \alpha(v_c) \rrbracket^-$, and if $T \in \llbracket v_a \rrbracket^-$ with $\gamma_{v_c}(T) = \perp$:*

- (1) Soundness: *for all $T_a \in \llbracket v_a \rrbracket^-$, if $\gamma_{v_c}(T_a) \neq \perp$ then $T_a \in \llbracket \varrho_T(v_a) \rrbracket^-$.*
- (2) Completeness: *$\llbracket \varrho_T(v_a) \rrbracket^- \subseteq \llbracket v_a \rrbracket^-$.*

In simple terms, this states that refinements only remove non concretisable counterexamples from the abstraction. In practice, it typically removes T_a itself, or a generalised set including T_a . Automated tools for protocol analysis typically display an attack trace T as a partial order on a set of actions: our implementation for example makes ϱ_T exclude all linearisations of the dependency graph of T (Section 4.2). Combining all these properties, we obtain the correctness of the overall verification pipeline.

THEOREM 4.4 (CORRECTNESS OF ALGORITHM 1). *Let v_c be a concrete instance and assume that Propositions 4.1 to 4.3 hold. Provided $\text{verify}(v_c)$ terminates:*

- (1) *If $\text{verify}(v_c) = \text{accept}$ then $\llbracket v_c \rrbracket^- = \emptyset$.*
- (2) *If $\text{verify}(v_c) = \text{reject}_{T_c}$ then $T_c \in \llbracket v_c \rrbracket^-$.*

PROOF. If $i \in \{1, 2\}$, we write $\Gamma_i(v_c, \alpha)$ the statement of Proposition 4.2 Item i . Given an abstraction β , we then call $\Gamma(v_c, \beta)$ the conjunction of $\Gamma_1(v_c, \beta)$, $\Gamma_2(v_c, \beta)$, and the statement $\llbracket \beta(v_c) \rrbracket^- \subseteq \llbracket \alpha(v_c) \rrbracket^-$. We first prove the following property, referred to as $(\star)$:

Let v_c, β such that $\Gamma(v_c, \beta)$. If $\mathcal{O}(\beta(v_c)) = \text{reject}_T$ with $\gamma_{v_c}(T) = \perp$, we have $\Gamma(v_c, \varrho_T \circ \beta)$.

Indeed, let v_c, β, T satisfying the assumptions of $(\star)$. By oracle's completeness (Proposition 4.1, Item 2), we have $T \in \llbracket \alpha(v_c) \rrbracket^-$. By refinement's completeness (Proposition 4.3, Item 2, with $v_a := \beta(v_c)$),

we have $\llbracket \varrho_T \circ \beta(v_c) \rrbracket^- \subseteq \llbracket \alpha(v_c) \rrbracket^-$. Then $\Gamma_1(v_c, \varrho_T \circ \beta)$ easily follows from $\Gamma_1(v_c, \beta)$ and refinement's soundness (Proposition 4.3, Item 1, with $v_a := \beta(v_c)$), while $\Gamma_2(v_c, \varrho_T \circ \beta)$ follows from $\Gamma_2(v_c, \beta)$.

We then deduce from $(\star)$ and a straightforward induction that, in a computation of $\text{verify}(v_c)$ in Algorithm 1, for all recursive calls made to $\text{cegar}(v_c, v_a)$, there is an abstraction β such that $v_a = \beta(v_c)$ and $\Gamma(v_c, \beta)$. We then easily deduce Item i , $i \in \{1, 2\}$, of the theorem by oracle correctness (Proposition 4.1, Item i) and $\Gamma_i(v_c, \beta)$. $\square$

4.2 Implementation of the Interface

We now describe the effective construction of the previous theoretical interface, reflecting our tool implementation. We first define:

Definition 4.5 (Numeric atoms, Unsupported atoms, Abstract satisfiability). Consider the atomic formulae that are either of the form $u = v$, when either u or v has a numeric subterm, or of the form $u > 0$; we call them the *numeric atoms*. In the rest of the paper, we assume a fixed subset of numeric atoms that we call *unsupported atoms*. We then define *abstract satisfiability* $\models_{\mathbb{A}}^a$

$$T \models_{\mathbb{A}}^a \varphi \text{ iff } T \models_{\mathbb{A}} \bar{\varphi}$$

where $\bar{\varphi}$ is obtained by replacing all unsupported atoms in φ by the trivial formula $\top$ (resp. the contradiction $\perp$) at negative (resp. positive) polarity, namely, when the atom is under an odd (resp. even) number of implication ($\implies$) left-hand sides.

Unsupported atoms are intuitively the quantitative features that the back-end tool does not support. This typically includes arithmetic constraints, by opposition to broadly-supported qualitative features formalised with numeric terms such as ordering constraints $i < j$. Keeping the notion of unsupported atom abstract helps keeping the presentation simple, and also more general.

Definition 4.6 (Instances and semantics). A concrete instance $v_c \in V_c$ is a verification instance in the sense of Section 3.3. An abstract instance $v_a \in V_a$ is a concrete instance $(\mathbb{A}, P, \varphi)$ where $\mathbb{A}$ and P have no numeric variables, timestamps $@t$, nor “when” instructions.

Given $v = (\mathbb{A}, P, \varphi)$ a concrete or abstract instance, its semantics $\llbracket v \rrbracket$ is defined as the set of $\mathbb{A}$ -traces of P (Definition 3.5). The counterexample semantics are then defined as:

$$\llbracket v_c \rrbracket^- = \{T \in \llbracket v_c \rrbracket \mid T \not\models_{\mathbb{A}} \varphi\} \quad \llbracket v_a \rrbracket^- = \{T \in \llbracket v_a \rrbracket \mid T \not\models_{\mathbb{A}}^a \varphi\}.$$

Oracle. By design, oracles $\mathcal{O}$ are to be implemented by off-the-shelf tools in regular protocol analysis such as [6, 10]. Such tools often have a syntax for trace properties similar to Definition 3.6, and provide the required oracle correctness (Proposition 4.1). We do not make our presentation specific to the Tamarin back-end language, by abstracting away the calculus-to-Tamarin translation conducted by Sapic [27]. This can be seen as an internal machinery to manage the interaction with the back-end tool, whose correctness directly stems from the proofs of [27]. We will therefore proceed as if $\mathcal{O}$ was supporting a non-quantitative variant of our language, i.e., the abstract instance as defined above.

One point of contention is nonetheless atomic composition ($::$) which, although conceptually simple, is not standard. Starting from a specification language similar to ours, Sapic translates into the languages of other tools to use them as back-end provers (historically, Tamarin, but several other tools are supported today [18]). In

the case of Tamarin, conceptually speaking, two instructions $i_1; i_2$ of a process will be translated into two Tamarin rules

$$[L_1] - [E_1] \rightarrow [R_1] \quad \text{and} \quad [L_2] - [E_2] \rightarrow [R_2]$$

where L_1, R_1, L_2, R_2 are defined in our framework as multisets of terms, and E_1, E_2 as sets of events. Each rule $[L_i] - [E_i] \rightarrow [R_i]$ can be interpreted as a transition consuming the elements of L_i from a global execution state S , then adding the elements of R_i to S in exchange, and labelling the overall operation with the events of E_i . In particular, the atomic composition $i_1 :: i_2$ is easily encoded by:

$$[L_1 \cup (L_2 \setminus R_1)] - [E_1 \cup E_2] \rightarrow [(R_1 \setminus L_2) \cup R_2].$$

In practice however, a few features such as synchronous communications exhibit a translation involving more than one Tamarin rule per branch of the instruction. Due to the incompatibility with the above approach, we exclude them from our tool. This also requires natural instruction orderings, such as inputs having to appear before outputs in atomic blocks, or unfound before pull before push. We implemented the rest of the Sapic-to-Tamarin translation, with the above straightforward support for atomic operations. For simplicity, our prototype relies on the legacy translation of Sapic [27], as its more recent developments [18] introduce optimisations making the correctness of our support for atomicity less self-evident.

Note in particular that our presentation is backend-agnostic, in that the construction of $\mathcal{O}$ may swap Tamarin out for any other symbolic tool extended with atomic composition.

Abstraction. Our abstraction α intuitively substitutes numeric terms $x \in \mathbb{R}$ by plain, uninterpreted terms $\alpha_{\text{term}}(x)$. It also marks every block of atomic instruction with an explicit position and a timestamp, for the purpose of tracking back lost quantitative information. Each event Ev/n is notably replaced by an alternate $\text{Ev}_a/n + 1$ taking the current time as an additional argument. This is how we encode the use of formula timestamps as plaintext arguments (as in, e.g., $\mathbf{A}@\mathbf{i} \Rightarrow \mathbf{B}(\mathbf{i})@j$). Formally, given $v_c = (\mathbb{A}, P, \varphi)$ with $\mathbb{A} = (\mathcal{F}, E)$, we define $\alpha(v_c) = (\mathbb{A}', P', \varphi')$, with $\mathbb{A}' = (\mathcal{F}', E')$ and:

$$\begin{aligned} \mathcal{F}' &= \mathcal{F} \uplus \{\text{numeric}/1\} & E' &= \{(\alpha_{\text{term}}(u), \alpha_{\text{term}}(v)) \mid (u, v) \in E\} \\ \varphi' &= \alpha_{\text{form}}(\varphi) & P' &= \text{in}(\text{numeric}(t_\varepsilon)) :: \alpha_{\text{proc}}(P, \varepsilon, t_\varepsilon) \end{aligned}$$

where α_{term} , α_{form} and $\alpha_{\text{proc}}(P, p, a)$ are defined in Figure 5, where p is a position (i.e., a sequence of integers, ε referring to the empty sequence), $(t_p)_p$ a family of fresh non-numeric variables indexed by positions, and a is a sequence of variables. The construction assumes, for each position p , an event State_p of suitable arity, and an event $\text{Ev}_a/n + 1$ for each event Ev/n appearing in P . When later justifying the abstraction's completeness (Proposition 4.2, Item 2), we will however require an additional structural assumption on P :

Assumption 4.1 (Compartmentation). *The process P is compartmented, namely, all its numeric subterms are either of the form $\text{numeric}(x)$ for some variable x , or occur inside a when condition.*

This intuitively ensures that the process P only carries either trivial, or “when”-guarded numeric information. This assumption is however without loss of generality, as it suffices to replace P by $\alpha_{\text{proc}}^N(P)$, which is the syntactic pre-processing defined similarly as

in Section 2, by transforming each atomic block β of P into:

$$\begin{aligned} &\text{in}(\text{numeric}(x_1)) :: \dots :: \text{in}(\text{numeric}(x_n)) :: \\ &\text{when } x_1 = u_1 :: \dots :: \text{when } x_n = u_n :: \beta' \end{aligned}$$

where

- (1) $x_1, \dots, x_n \in \text{vars}(\beta')$ are fresh variables;
- (2) replacing occurrences of $\text{numeric}(x_i)$ by u_i in β' yields β ;
- (3) the u_i are all numeric terms;
- (4) β' does not contain numeric values apart from numeric variables or in when instructions.

Concretisation. Given a concrete instance $v_c = (\mathbb{A}, P, \varphi)$, let us now define the concretisation function γ_{v_c} . The construction is however properly defined and correct only when v_c satisfies additional properties, that are therefore required by our implementation.

Assumption 4.2 (Type-homogeneity). *The formula φ is type homogeneous, namely, in all of its unsupported atoms $u = v$ or $u > 0$, both u and v are numeric terms.*

This assumption will ensure that the concretisability of an abstract trace can be reduced to checking the validity of systems of arithmetic constraints. Also, we impose a crucial, albeit standard, restriction on quantified variables:

Assumption 4.3 (Guardedness). *The abstraction $\varphi_a = \alpha_{\text{form}}(\varphi)$ is guarded, namely, it satisfies the following property. Let $\varphi_0 = \forall x_1, \dots, x_n. \varphi_1$ be a sub-formula of φ_a , with no quantifiers occurring at the first position before φ_0 in φ_a , nor at the first position in φ_1 . Then there exist $p \geq 1$ and formulae $\psi_1, \dots, \psi_p, \psi$ such that*

$$\varphi_1 = \left(\bigwedge_{j=1}^p \psi_j \right) \Rightarrow \psi$$

where for all $i \in \llbracket 1, n \rrbracket$ such that $x_i \in \text{vars}(\varphi_1)$, there is $j \in \llbracket 1, p \rrbracket$ such that $\psi_j = \text{Ev}(\vec{u})@t$ and $x_i \in \{\vec{u}, t\}$ or $\text{numeric}(x_i) \in \{\vec{u}\}$.

This notion is largely standard when not accounting for numeric features [6], and simply ensures that all valid instantiations of a quantified variable $\forall x$ can be nailed down to a finite number per trace. The above assumption lifts this notion to the quantitative in the natural way, and imposes it on the abstract formula φ_a that is sent to the back-end oracle. More formally, the consequence of guardedness we need is the following.

LEMMA 4.7. *Let T be a trace and φ_0 be a formula that matches the notations and hypotheses of Assumption 4.3.*

Then there exists a finite set S of pairs (σ, c) such that:

- (1) σ is a substitution of domain $\text{dom}(\sigma) \subseteq \{x_1, \dots, x_n\}$;
- (2) c is a boolean formula over arithmetic constraints whose variables range over $\text{vars}(c) \subseteq \{x_1, \dots, x_n\} \cap X^N$;
- (3) $T \models_{\mathbb{A}} \varphi_0$ iff $T \models_{\mathbb{A}} \bigwedge_{(\sigma, c) \in S} (c \Rightarrow \varphi_1 \sigma)$.

This is a standard characterisation of guarded formulae in the field, the arithmetic side conditions c being the only addition accounting for numeric values; we refer to the early presentation of Tamarin's theory [34, Section 8.2] for a reference proof.

Under these assumptions, we write $\alpha(v_c) = (\mathbb{A}_a, P_a, \varphi_a)$ and consider an abstract counterexample $T_a \in \llbracket \alpha(v_c) \rrbracket^-$, in order to define the concretisation $\gamma_{v_c}(T_a) \in \llbracket v_c \rrbracket \cup \{\perp\}$. We express the concretisability of T_a as a collection of arithmetic constraints characterising its executability in a straightforward way. Since the representation

$$\begin{aligned}
\alpha_{\text{term}}(n) &= \bar{n} \quad \text{if } n \in \mathbb{R}, \text{ and with } \bar{n} \in \mathcal{N}_{\text{pub}} \setminus \mathbb{R} \text{ fresh} \\
\alpha_{\text{term}}(t) &= \text{numeric}(\bar{t}) \quad \text{if } t \in \mathcal{X}^N, \text{ and with } \bar{t} \in \mathcal{X} \setminus \mathcal{X}^N \text{ fresh} \\
\alpha_{\text{term}}(a) &= a \quad \text{if } a \in (\mathcal{X} \cup \mathcal{N}) \setminus (\mathcal{X}^N \cup \mathbb{R}) \\
\alpha_{\text{term}}(f(u_1, \dots, u_n)) &= f(\alpha_{\text{term}}(u_1), \dots, \alpha_{\text{term}}(u_n)) \\
\alpha_{\text{proc}}(0, p, a) &= 0 \\
\alpha_{\text{proc}}(P_0 \mid P_1, p, a) &= \text{State}_p(a) :: (P'_0 \mid P'_1) \quad \text{if } P'_i = \text{in}(\text{numeric}(t_{p \cdot i})) :: \alpha_{\text{proc}}(P_i, p \cdot i, a \cdot t_{p \cdot i}) \\
\alpha_{\text{proc}}(!P, p, a) &= \text{State}_p(a) :: !\text{in}(\text{numeric}(t_{p \cdot 0})) :: \alpha_{\text{proc}}(P, p \cdot 0, a \cdot t_{p \cdot 0}) \\
\alpha_{\text{proc}}(\text{either } S_1 \text{ or } S_2, p, a) &= \text{either } \alpha_{\text{proc}}(S_1, p \cdot 0, a) \text{ or } \alpha_{\text{proc}}(S_2, p \cdot 1, a) \\
\alpha_{\text{proc}}(\text{skip}; P, p, a) &= \text{State}_p(a) :: \text{skip}; \text{in}(\text{numeric}(t_{p \cdot 0})) :: \alpha_{\text{proc}}(P, p \cdot 0, a \cdot t_{p \cdot 0}) \\
\alpha_{\text{proc}}(@t :: S, p, a) &= \alpha_{\text{proc}}(S[t \mapsto \text{numeric}(t_p)], p, a) \\
\alpha_{\text{proc}}(\text{when } c :: S, p, a) &= \text{SMT}(\alpha_{\text{term}}(c)) :: \alpha_{\text{proc}}(S, p, a) \\
\alpha_{\text{proc}}(\text{in}(u) :: S, p, a) &= \text{in}(\alpha_{\text{term}}(u)) :: \alpha_{\text{proc}}(S, p, a \cdot x_1 \dots x_n) \quad \text{if } \text{vars}(\alpha_{\text{term}}(u)) = \{x_1, \dots, x_n\} \\
\alpha_{\text{proc}}(\text{out}(u) :: S, p, a) &= \text{out}(\alpha_{\text{term}}(u)) :: \alpha_{\text{proc}}(S, p, a) \\
\alpha_{\text{proc}}(\text{new } k :: S, p, a) &= \text{new } k :: \alpha_{\text{proc}}(S, p, a \cdot k) \\
\alpha_{\text{proc}}(\text{push } u :: S, p, a) &= \text{push } \alpha_{\text{term}}(u) :: \alpha_{\text{proc}}(S, p, a) \\
\alpha_{\text{proc}}(\text{pull } u :: S, p, a) &= \text{pull } \alpha_{\text{term}}(u) :: \alpha_{\text{proc}}(S, p, a \cdot x_1 \dots x_n) \quad \text{if } \text{vars}(\alpha_{\text{term}}(u)) = \{x_1, \dots, x_n\} \\
\alpha_{\text{proc}}(\text{unfound } u :: S, p, a) &= \text{unfound } \alpha_{\text{term}}(u) :: \alpha_{\text{proc}}(S, p, a) \\
\alpha_{\text{proc}}(\text{Ev}(\bar{u}) :: S, p, a) &= \alpha_{\text{term}}(\text{Ev}_a(\bar{u}, \text{numeric}(t_p))) :: \alpha_{\text{proc}}(S, p, a) \\
\alpha_{\text{form}}(u_0 = u_1) &= (\bar{u}_0 = \bar{u}_1) \quad \text{where } \bar{u}_i = \alpha_{\text{term}}(u_i) \text{ if } (u_0 = u_1) \text{ is an unsupported atom, and } \bar{u}_i = u_i \text{ otherwise} \\
\alpha_{\text{form}}(u > 0) &= (\bar{u} > 0) \quad \text{where } \bar{u} = \alpha_{\text{term}}(u) \text{ if } (u > 0) \text{ is an unsupported atom, and } \bar{u} = u \text{ otherwise} \\
\alpha_{\text{form}}(\text{Ev}(\bar{u})@t) &= \alpha_{\text{term}}(\text{Ev}_a(\bar{u}, \alpha_{\text{term}}(t)))@t \\
\alpha_{\text{form}}(\text{K}(u)@t) &= \text{K}(\alpha_{\text{term}}(u))@t \\
\alpha_{\text{form}}(\varphi \Rightarrow \psi) &= \alpha_{\text{form}}(\varphi) \Rightarrow \alpha_{\text{form}}(\psi) \\
\alpha_{\text{form}}(\forall t. \varphi) &= \forall \bar{t}. \bar{t}. \alpha_{\text{form}}(\varphi) \quad \text{if } t \in \mathcal{X}^N \text{ and } \alpha_{\text{term}}(t) = \text{numeric}(\bar{t}) \\
\alpha_{\text{form}}(\forall x. \varphi) &= \forall x. \alpha_{\text{form}}(\varphi) \quad \text{if } x \in \mathcal{X} \setminus \mathcal{X}^N
\end{aligned}$$

Figure 5: Definitions of abstractions α_{term} , α_{proc} , α_{form}

of traces may vary from one back-end tool to another, we introduce a generic notion of dependency graph to capture most representations of T_a tools may use. A node of such a graph is intuitively a rule application of the operational semantics, namely:

Definition 4.8 (Node, ts). A *node* is an instance of one of the rules of the semantics (Rules (LIFT), (DEDUCE), or (ATK), Figure 3), stripped of the extended processes it relates. Namely, the node records the choice of the rule together with the data instantiating its premises (the selected process, the substitutions, fresh names and consumed terms...). Given $r \in \mathbb{R}_+^*$, we then write

$$A \xrightarrow[r]{\vartheta} A'$$

when ϑ is applicable to A and yields A' at time r , i.e., when $A \xrightarrow{\bar{w}} A'$ is the corresponding instance of Figure 3 and r is the timestamp of A' . Each node is attached a numeric variable ts_ϑ , as follows. By construction of α (Figure 5), ϑ may be an instance of Rule (LIFT) emitting one event $\text{State}_{p_i}(\bar{u}_i)$, and $\bar{u}_i$ then contains the name ts_i that instantiated the variable t_{p_i} (notations of the definition of α_{proc}); in this case $\text{ts}_\vartheta = \text{ts}_i$. Otherwise, ts_ϑ is a fresh variable.

Definition 4.9 (Dependency graph, linearisation). A *dependency graph* $G(T_a)$ of an abstract trace T_a is a directed acyclic graph with nodes in the sense of Definition 4.8, and subject to the restrictions below. A *linearisation* of $G(T_a)$ is an injection $\tau : \{\text{ts}_\vartheta \mid \vartheta \text{ node of } G(T_a)\} \rightarrow \mathbb{R}_+^*$ such that $\text{ts}_\vartheta \tau < \text{ts}_{\vartheta'} \tau$ for all edges $\vartheta \rightarrow \vartheta'$ of $G(T_a)$. We then require that $G(T_a)$ satisfies the following properties for all linearisations τ . Writing $\vartheta_1, \dots, \vartheta_n$ the nodes of $G(T_a)$

ordered by increasing value of $\text{ts}_{\vartheta_i} \tau$, and A_0 the first extended process of the trace T_a , there exists a valid (abstract) trace of the form

$$A_0 \xrightarrow[\text{ts}_{\vartheta_1} \tau]{\vartheta_1} A_1 \cdots \xrightarrow[\text{ts}_{\vartheta_n} \tau]{\vartheta_n} A_n$$

for some extended processes $A_1, \dots, A_n$. By abuse of notation, this trace, written $G(T_a)\tau$, is also called a *linearisation* of $G(T_a)$. We require that T_a is a linearisation of $G(T_a)$.

This notion is purposely abstract so as not to commit to a specific output format for the back-end counterexamples, should it be an actual graph (as Tamarin provides) or a flat sequence.

The concretisation is then expressed over variables $\tau_u \in \mathbb{R}_+^*$, one for each (numeric) term u . Intuitively, τ_u is the real number the placeholder u stands for: the time at which the corresponding step is performed, or the numeric value the adversary provided. Accordingly, given an abstract term or formula ψ , we write $\lceil \psi \rceil$ for the arithmetic expression obtained by replacing in ψ every subterm $\text{numeric}(u)$ by τ_u , every name $\alpha_{\text{term}}(n)$, $n \in \mathbb{R}$, by n , and every timepoint of T_a by $\tau_{\text{ts}_\vartheta}$, with ϑ the node acting at that time.

Definition 4.10 (Trace constraints). The *trace constraint* of T_a is the formula $\Gamma_{\text{ord}}(T_a) \wedge \Gamma_{\text{when}}(T_a)$ where:

$$\begin{aligned}
\Gamma_{\text{ord}}(T_a) &= \bigwedge_{\substack{\vartheta \rightarrow \vartheta' \\ \text{edge of } G(T_a)}} \tau_{\text{ts}_\vartheta} < \tau_{\text{ts}_{\vartheta'}} \wedge \bigwedge_{\substack{\vartheta, \vartheta' \text{ distinct} \\ \text{nodes of } G(T_a)}} \tau_{\text{ts}_\vartheta} \neq \tau_{\text{ts}_{\vartheta'}} \\
\Gamma_{\text{when}}(T_a) &= \bigwedge_{\substack{\text{SMT}(c) \\ \text{occurs in } T_a}} \lceil c \rceil
\end{aligned}$$

Note that T_a is executable in the concrete instance *iff* the trace constraints are satisfied, due to Assumption 4.1, which justifies the abstraction's completeness (Proposition 4.2, Item 2). The direct implication follows from T_a being a linearisation of $G(T_a)$, whose timestamps therefore satisfy $\Gamma_{\text{ord}}(T_a)$, and the converse one from the requirement of Definition 4.9. The abstraction's soundness on the other hand (i.e., the Item 1), follows from a straightforward induction on the trace T_c . In addition to it, we define the constraints induced by the abstract formula φ_a . Intuitively, it collects back the unsupported atoms that were ignored in the abstract instance due to the partial predicate $\models_a^\perp$.

Definition 4.11 (Formula constraints). With the above notations, the *formula constraint* of T_a against a subformula φ of φ_a , written $\Gamma_{\text{form}}(T_a, \varphi)$, is:

- If $\varphi = (u = v)$ is not a numeric atom, $\Gamma_{\text{form}}(T_a, \varphi) = \top$ if $T_a \models_a \varphi$, and $\Gamma_{\text{form}}(T_a, \varphi) = \perp$ otherwise.
- If φ is a numeric atom, $\Gamma_{\text{form}}(T_a, \varphi) = \lceil \varphi \rceil$.
- If $\varphi = \text{Ev}_a(\vec{u}, \text{numeric}(v))@t$, $\Gamma_{\text{form}}(T_a, \varphi) = (\tau_v = \tau_{\text{ts}_\varphi})$ if $T_a \models_a \varphi$, writing ϑ for the node of T_a acting at time t , and $\Gamma_{\text{form}}(T_a, \varphi) = \perp$ otherwise.
- If $\varphi = K(u)@t$, $\Gamma_{\text{form}}(T_a, \varphi) = \top$ if $T_a \models_a \varphi$, and $\Gamma_{\text{form}}(T_a, \varphi) = \perp$ otherwise.
- $\Gamma_{\text{form}}(T_a, \varphi \Rightarrow \psi) = \Gamma_{\text{form}}(T_a, \varphi) \Rightarrow \Gamma_{\text{form}}(T_a, \psi)$.
- Let $\varphi_0 = \forall x_1, \dots, x_n. \varphi_1$ following the notations and hypotheses of Assumption 4.3. If we let S the set given by Lemma 4.7:

$$\Gamma_{\text{form}}(T_a, \varphi_0) = \bigwedge_{(\sigma, c) \in S} \Gamma_{\text{form}}(T_a, c \Rightarrow \varphi_1 \sigma)$$

Note that numeric atoms are systematically turned into arithmetic constraints, including when they are supported: evaluating such an atom on T_a would be unsound, as $\Gamma_{\text{ord}}(T_a)$ only requires the concrete timestamps to follow the edges of $G(T_a)$, and not the whole interleaving of T_a .

Note that whenever required in the above cases, checking $T_a \models_a \varphi$ is either trivial, or can be encoded as a simple call to the oracle O . The overall concretisation $\gamma_{v_c}(T_a)$ is then defined as follows, writing

$$\Gamma = \Gamma_{\text{ord}}(T_a) \wedge \Gamma_{\text{when}}(T_a) \wedge \neg \Gamma_{\text{form}}(T_a, \varphi_a)$$

Since Γ is a boolean formula over arithmetic atoms by Assumption 4.2, we talk about Γ being *satisfiable* in the usual sense: namely, if there exists a substitution σ such that $\Gamma \sigma$ is a tautology.

- (1) If Γ is unsatisfiable, we define $\gamma_{v_c}(T_a) = \perp$.
- (2) If $\Gamma \sigma$ is a tautology, σ induces a linearisation $\tau_\sigma : \text{ts}_\vartheta \mapsto \tau_{\text{ts}_\vartheta} \sigma$ of $G(T_a)$, by the disequations of $\Gamma_{\text{ord}}(T_a)$. We then let $\gamma_{v_c}(T_a) = T_c$, with T_c the trace of P applying the same rules of the operational semantics as $G(T_a) \tau_\sigma$, the terms ξ deduced by Rule (DEDUCE) being replaced by $\lceil \xi \rceil \sigma$.

Refinement. With the same notations and $\gamma_{v_c}(T_a) = \perp$, we let σ be a bijection mapping all fresh names of T_a , namely $\text{names}(T_a) \setminus \text{names}(P_a)$, to fresh variables. We then define the following restriction excluding all instances of $T_a \sigma$ from the analysis, i.e., T_a and all variants substituting fresh names (which can be seen as term placeholders in traces) by any other term. We write $\text{vars}(T_a \sigma) = \{x_1, \dots, x_p\}$ and $\text{State}_{p_i}(\vec{u}_i)@t_i$, $i \in \llbracket 1, n \rrbracket$, the set of **State** events appearing in $T_a \sigma$, emitted by the respective nodes $\vartheta_1, \dots, \vartheta_n$ of $G(T_a)$,

and where the t_i are fresh variables. We also define:

$$\text{Excl}(T_a) = \forall (x_i)_{i=1}^p, (t_i)_{i=1}^n. \bigwedge_{i=1}^n \text{State}_{p_i}(\vec{u}_i)@t_i \Rightarrow \bigvee_{\substack{\vartheta_i \rightsquigarrow \vartheta_j \\ \text{in } G(T_a)}} t_i \geq t_j$$

where $\vartheta \rightsquigarrow \vartheta'$ when $G(T_a)$ has a path from ϑ to ϑ' whose intermediate nodes emit no **State** event, that is, the dependencies of T_a that its **State** events can express: the remaining nodes, typically instances of Rules (DEDUCE) and (ATK), carry no **State** timestamp. Observe that $\text{Excl}(T_a)$ is guarded in the sense of Assumption 4.3, the x_i occurring in no **State** event being unused, and that its ordering atoms are expected to be supported, as they are by any back-end providing temporal ordering: were they unsupported, $\models_a^\perp$ would trivialise the restriction. We then simply define the refinement as:

$$\mathcal{Q}_{T_a}((\mathbb{A}_a, P_a, \varphi_a)) \triangleq (\mathbb{A}_a, P_a, \text{Excl}(T_a) \Rightarrow \varphi_a).$$

We implement this refinement using a Tamarin *restriction*, a feature to restrict the analysis to the set of traces satisfying, here, $\text{Excl}(T_a)$. The refinement's correctness (Proposition 4.3) then follows from the fact that the **State** events of a trace determine its Γ_{when} and Γ_{form} constraints, so that a trace matching them while respecting $\rightsquigarrow$ has an unsatisfiable trace constraint as well.

5 Experiments

In this section, we evaluate our implementation on concrete protocols involving quantitative components. In addition to the minimal theoretical core presented in this paper, the tool also implements a large range of other features (e.g., assertions, axioms, pattern matching, and records) to facilitate the specification of large models. We recall that the tool and experimental evaluation are available at

<https://gitlab.com/irakoton/tidy-automation>

The results are summarised in Figure 6 and detailed in this section.

Protocol		Status	#refin.
Motivating example (Section 2)		✓ < 1 s	1
String sampling (with time commitments)		✓ 2 s	1
Sealed-bid auction		✓ 2 s	2
Multi-round beacon	1 round	✓ 2 s	3
	2 rounds	✓ 30 s	11
Unicorn	1 honest	✓ < 1 s	2
	1 honest + 1 dishonest	✓ 6 s	6
Fair signing	1 round	✓ < 1 s	1
	2 rounds	✓ 5 s	4
	3 rounds	✓ 1 m 17 s	15
UTXO swap	initiator, V1	⚡ 1 m 11 s	0
	initiator, V2	⚡ 3 m 10 s	0
	initiator, V3	⊗	
✓ proved ⚡ valid attack ⊗ timeout (> 20 min)			

Figure 6: Summary of the experimental results (end-to-end runtime). The last column counts invalid attacks.

5.1 Protocols Based on Timed Cryptography

As first case studies, we verified examples of protocols based on timed cryptography. Primitives from timed cryptography provide guarantees concerning their execution time, in the same vein as the VDF of Section 2. Besides VDF, we include here examples built from time commitments [12] and verifiable timed signatures [41].

Protocols building on some of these primitives have previously been studied in the symbolic model [5], with an approach similar in spirit to ours. However, in our terminology, all of their CEGAR steps except the oracle call were performed manually as a form of general methodology rather than a fully automated pipeline. As such, the models we verify with our tool can be significantly more complex and less error prone. A failure to encode a fair-contract signing example that we verify is for instance reported in [5]. This specific protocol indeed involves an inherently quantitative security notion making it tedious to approximate by hand properly: two distrustful participants keep exchanging successive locked signatures, dividing by 2 the forcing cost at each round. Although one party may abort mid-round, the fairness claim is that the other party is only worse off by at most a factor of 2 in terms of computation effort. We also verify Unicorn [28] which can be seen as the more complex, multi-party generalisation of the simplified motivating example of Section 2, as well as other classical protocols from the literature [11, 12].

5.2 Atomic Swap

As a more complex case study, we analysed the atomic swap of assets across blockchains, as proposed in [40]. For the purpose of scalability, we focused in our experiments on a lighter model with 1-to-1 transactions and a single blockchain, but will describe the general version of the protocol here for clarity.

Two parties **A** (initiator) and **B** (responder), who own assets on two different blockchains, wish to exchange them without trusting each other nor a third party. The security property of interest is fairness, namely that either both parties receive the other's asset, or neither of them does. This protocol is claimed to be secure by means of a cryptographic analysis in the UC framework, a computational model of cryptography [16]. While this fact is sometimes lightly challenged for a malicious **A**, e.g., a recent work [36] claiming a fairness violation where **A** can end up with both assets, scenarios involving a dishonest **B** are rarer, to the best of our knowledge. As such, the protocol is still widely believed to be secure against a malicious **B**. This motivated our analysis of this side of the protocol, which reported two genuine attack vectors on a model derived from the description of [40]. We discuss their implications in the rest of this section, but mention now that they rather come from occasional underspecification of a phase in the protocol description, than from a fundamental flaw in its theoretical design.

Blockchain Model. Part of the complexity of atomic swap protocols is directly inherited from the underlying blockchain model, which requires reflecting functional properties of the ledger consensus among mutually suspicious parties. We modelled here a UTXO blockchain, i.e., assets that can be seen as discrete nodes moving in ownership accordingly to the payload of transactions published in a shared global state. Transactions may include here *timelocks*, which is a numeric value indicating a minimum absolute time before which the enclosing transaction will never be finalised.

To outline the blockchain model, we include two main events **Submit**(*tx*) and **Publish**(*tx*), respectively sending and finalising a transaction on the network. Their structural properties are then enforced by some process mechanisms or axioms, and read as:

- (1) Published transactions have a unique identifier (hash) in the ledger and are well signed.
- (2) A publication at time t with timelock s is so that $t \geq s$.
- (3) Two published transactions cannot spend the same input.
- (4) Liveness: for any submission at time t attempting to spend an asset x , there is a published transaction in the interval $[0, t + \Delta]$ spending x —in practice either the submission in question, a racing transaction, or an earlier one.

Here, Δ is an abstract parameter representing a conservative upper bound of the blockchain publication time. The correct setup of timelocks from the value of Δ can be a key to this protocol (in)security, hence the appeal of quantitative tools.

Atomic Swap Overview. We next overview the general workings of atomic swaps. Consider two parties, **A** and **B**, that respectively own **coin_A** in *chain_A* and **coin_B** in *chain_B*. We still highlight that our own model file is written in a single-chain setting to simplify the protocol's flow, as a first scalability test for our prototype.

The protocol proceeds as follows: In a *locking phase*, **A** and **B** lock their coins into vaults from where they can either be refunded to their owner, or transferred to the other party, upon submission of well-defined transactions. To achieve fairness, the validity of the two claim transactions **ctx_B** and **ctx_A** are conditioned on the knowledge of a cryptographic secret x , which is initially known by the initiator **A**. If party **A** wants to publish **ctx_A** during the *execution phase* to claim the coins from **vault_{BA}**, they are also cryptographically bound to release the secret x , which can then be used by **B** to validate **ctx_B**.

If performed correctly, an atomic swap should broadly enforce a protocol execution in epochs as illustrated in Figure 7, namely:

- (e_0) **A** publishes **ltx_A** in *chain_A*, locking **coin_A** into **vault_{AB}**.
- (e_1) **B** publishes **ltx_B** in *chain_B*, locking **coin_B** into **vault_{BA}**.
- (e_2) **A** publishes **ctx_A** in *chain_B*. **A** obtains **coin_B** in *chain_B* and reveals secret x to **B**.
- (e_3) If secret x has been revealed, **B** publishes **ctx_B** in *chain_A*, effectively obtaining **coin_A** in *chain_A*. Alternatively, **B** can publish refund transaction **rtx_B** in *chain_B* to refund **coin_B** from **vault_{BA}**.
- (e_4) If **ctx_A** didn't happen, **A** publishes refund transaction **rtx_A** in *chain_A*.

An aspect which is sometimes omitted from such descriptions of the atomic swap protocol is its *setup phase*, where the protocol parties exchange the protocol transactions and provide each other with authorisations for these transactions. This exchange is required because transactions that spend funds from vaults (such as the claim and refund transactions) need to be authorised by the owners of the vault. Since the ownership for **vault_{AB}** and **vault_{BA}** is shared between **A** and **B**, both parties need to provide authorisations for the claim and refund transactions during setup.

More concretely, the setup protocol is executed in turns as follows:

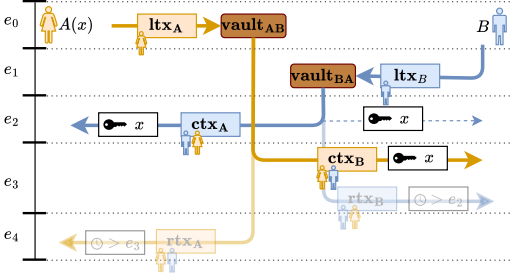


Figure 7: Atomic swap overview: claim (highlighted) and refund (in gray) transactions

- A sends vault_{AB} to B specifying the vault where A plans to lock coin_A during the posterior execution phase.
- B sends vault_{BA} to A specifying the vault where B plans to lock coin_B .
- A sends $(\text{ctx}_A, \text{rtx}_A)$ to B, effectively asking for B's authorisation on these transactions.
- B sends $(\text{ctx}_B, \text{rtx}_B)$ to A for their authorisation.
- A sends the authorisations for ctx_B and rtx_B to B. At this point, B can locally add their own authorisation for both transactions. Thus, both transactions are ready to be used by B once vault_{AB} and vault_{BA} are funded.
- B sends the authorisations for ctx_A and rtx_A to A, who can then similarly append their own authorisations to these two transactions.

Note that during the setup phase, no coin is locked yet in any of the vaults. Honest A (B, respectively) should ensure to only lock funds into vault_{AB} (vault_{BA}) provided that the setup was successful, and they obtained authorisations to publish ctx_A and rtx_A (ctx_B and rtx_B). Interestingly, it turns out that when the users miss subtle consistency checks before providing authorisation, the fairness of the whole protocol can be undermined. The two attack vectors reported by our tool are typical cases where A could be attacked in a scenario where they omit some checks on transactions. What makes these attack vectors particularly surprising is that they affect the user A, who, as the swap initiator holding the secret x at the start of the session, otherwise enjoys a privileged role in the protocol execution.

Recipient-mismatch attack. In the first attack, summarised in Figure 8, an honest user A gets tricked into initiating the swap without holding sufficient authorisation to execute the ctx_A transaction. A dishonest B, during setup, provides A with an inconsistent lock transaction $\text{ltx}_B : \text{coin}_B \rightarrow \text{vault}_{BB}$, which, instead of locking the coins into the jointly controlled vault vault_{BA} , locks those coins into a vault vault_{BB} controlled solely by B. Without an explicit check from A (which our initial protocol model did not have), A will unknowingly establish an invalid authorisation for ctx_A to spend coins from vault_{BB} (which would require two authorisations by B). When A then attempts to submit ctx_A , the dishonest B learns the secret x that permits B to publish ctx_B in epoch e_3 . As shown in Figure 6 (case initiator, V1), our tool automatically discovers this attack and validates that it breaks the atomicity of the swap

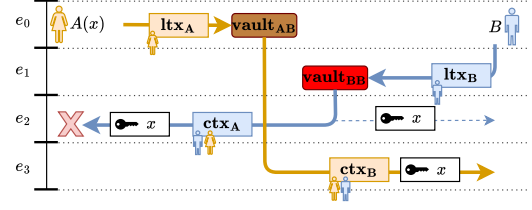


Figure 8: Recipient-mismatch attack

consistently with the quantitative constraints followed by A and by the blockchain.

The natural mitigation is for A to ensure during the setup phase that the recipient of ltx_B is a vault jointly controlled by A and B. We implemented this mitigation in our model of atomic swap, and after that, our tool reported the attack vector described below.

Vault-confusion attack. Even if A verifies that the recipient of ltx_B is a vault jointly controlled by A and B, B can still fool A into accepting a maliciously crafted setup. The idea is that during the setup phase, after receiving $\text{ltx}_A : \text{coin}_A \rightarrow \text{vault}_{AB}$ from A, malicious B provides A with $\text{ltx}_B := \text{ltx}_A$ (i.e., ltx_B being exactly the same as ltx_A). Intuitively, B is claiming that $\text{vault}_{BA} = \text{vault}_{AB}$. In this case, ltx_B locks B into an account that is jointly owned by A and B. Consequently, parties A and B, during setup, will continue exchanging authorisations on rtx_B , which, however, now spends the funds of vault_{AB} .

Such a poisoned setup allows the dishonest B to break the atomicity of the swap as follows. After A publishes ltx_A in epoch e_0 , B does not publish ltx_B in epoch e_1 . Note that at this point, A cannot submit rtx_A before epoch e_3 expires. Hence, as soon as epoch e_3 is reached, B publishes (the correctly authorised) rtx_B , spending the funds from A. To mitigate this attack, A would need to ensure during the setup phase that $\text{vault}_{AB} \neq \text{vault}_{BA}$.

Unlike the recipient-mismatch attack, we, however, believe this concrete scenario to be an artifact of our model. Due to scalability constraints, we actually modelled a lighter atomic swap protocol operating on a single blockchain. In practice, this attack may not be possible in a setting where users perform a swap across different chains since the vault representations may not be compatible between chains, naturally preventing $\text{vault}_{AB} = \text{vault}_{BA}$. We still believe that the described attack can be mounted when A takes part in two parallel cross-chain swaps with B. Then B can present vault_{AB}' from its second session with A as vault_{BA} in its first session. In this case, $\text{vault}_{AB}' = \text{vault}_{BA}$ would be possible since the chains of vault_{AB}' and vault_{BA} may have compatible representations (or even coincide). We acknowledge that either confirming or ruling out the feasibility of this attack in a multi-session setting requires non-trivial reasoning that our tool is not yet able to perform.

6 Related Work

Symbolic Approaches. Formal methods for the security of systems have been popular for quite some time. Even when sticking to the symbolic setting, tools like ProVerif [10], Tamarin [6] or SapiC [18] have become mature enough to verify industry-scale

protocols such as TLS 1.3 [8, 21], EMV [7], or even OPC UA [23]. There has also been work on playing beyond the limits of symbolic analysers, while retaining their characteristic automatability. One common example is the use of symbolic tools to verify distance-bounding protocols [14]. The goal of such protocols is to ensure that some agents have some physical proximity, such as in contact-less payments or for remote keys in automotive contexts. Symbolic approaches would typically extend existing models with notions of topology to be able to express physical proximity, and either reduce the verification problem to the standard setting, or simply extend the internal decision procedure of tools [22, 33].

Beyond Classical Models. Another common approach to retain some advantages of the symbolic model while mitigating its natural limitations is, intuitively, to *over-specify* the adversarial capabilities, to then *refine* it in case unrealistic attacks are exhibited. One typical example is [26], modelling signature schemes more permissively to capture real, subtle signature behaviours through an interaction of so-called *falsification* and *verification* models. Our approach of the quantitative features of protocols follows similar ideas, in that we over-approximate the protocol's behaviour by erasing such features, and use an automated refinement process to exclude invalid attacks.

In terms of theoretical framework, symbolic models including time components are relatively sparsely studied outside of the specific, unrelated case of distance-bounding protocols. Some early attempts include for example [29] that supports the verification of a very limited notion of security for real-time protocols, or more recently, the Tidy logics [5, 37]. The latter defines a more comprehensive extension of a classical symbolic framework (the applied-pi calculus) with real time, and a large class of security properties (hyperproperties). Despite proving some theoretical decidability results, we are not aware of tools based on similar ideas or models. For example, [5] only provides a CEGAR-like loop which, although similar in spirit to ours, is conducted informally and by hand, as opposed to our full formalisation and implementation. Meanwhile, [37] proves theoretical decidability results without implementation.

Similar Work. Because quantitative applications such as protocols based on timed cryptography have only recently emerged, they still lack in-depth academic scrutiny in standard formal models such as the symbolic framework. One may still mention work like [15] which models a public ledger in the Tamarin tool. Time is abstracted away using events, i.e., abstract, meta protocol actions representing the passage of time. Such approaches however fall short for the applications we consider, in that we consider intricate, sometimes nested quantitative conditions whose impact on the protocol flow is quite involved to characterise by hand. Other analyses have also arisen recently, e.g., [17], focusing on formalising emerging applications such as blockchain rollups. Despite relying less on logic in their framework, their formalisation shares a lot of conventions with symbolic models, in particular in the description of the network, adversarial capabilities, and cryptographic assumptions.

7 Conclusion

We presented a CEGAR framework for interfacing general-purpose security-protocol analysers as backends to support quantitative notions. We typically focused in our contributions on arithmetic

conditions solved by an external SMT solver, allowing us to automate the analysis of many applications involving, e.g., time cryptography or time-lock-based security designs. Due to the generality of the approach, we expect that the same pipeline could be used to verify other similar feature extensions of the applied pi calculus, and be a systematic way to integrate external solvers to existing tools soundly. Even when sticking to similar applications as the ones in this paper, a large range of protocols reliant on quantitative grounds remain to be explored. We may cite for example payment channels [2], virtual channels [3], coin mixing services [25], the BitVM family [31], and trustless bridges between L2s [35, 39].

Limitations and Future Work

Despite its conceptual simplicity, our CEGAR loop has proved its usefulness at tackling quantitative security by benefiting from off-the-shelf tools. However, our approach is not without limitations.

Experiments showed that we are often, as suggested by Figure 6, practically limited to a bounded number of sessions. This setting has its interest for an attack-search tool, in that most real attacks rarely exceed a few sessions anyway. Still, it makes the whole approach unfit to effectively prove unbounded quantitative properties. The reason is that we roughly exclude attacks one by one, which tends to generate an infinite number of false alarms for unbounded sessions. In the bounded setting (which, we recall, does not imply a finite number of *adversarial* executions), the number of invalid attacks is more likely to be finite. Even then however, their number tends to artificially explode as the number of sessions grows, e.g., exhibiting clusters of invalid attacks that only differed one from another in minor aspects. A counterexample-generalisation step during the refinement would likely bring significant improvements.

More generally, industrial-scale applications remain challenging. Typically, the full atomic swap (multi sessions, cross-chain, multi-IO transactions) is still out of reach. This prevented us for example from validating or invalidating the attacks and mitigations recently presented in [36]. This is likely a case where black-box reliance, despite its advantages, weighs heavily on the back-end due to the size of the translated model and its unusual patterns. A native (fork of a symbolic) analyser could be another research direction.

Acknowledgments

This work has been supported by the Heinz Nixdorf Foundation through a Heinz Nixdorf Research Group (HN-RG) and funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany's Excellence Strategy – EXC 2092 CASA – 390781972. The work is also part of the PICOCRYPT project that has received funding from the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation programme (Grant agreement No. 101001283) and is partially supported by Grant Excelencia Maria de Maeztu CEX2024-001471-M and the grant PID2022-142290OB-I00, funded by MICIU/AEI/10.13039/501100011033.

This paper was edited for grammar with Grammarly and Copilot.

References

- [1] Martin Abadi, Bruno Blanchet, and Cédric Fournet. 2018. The Applied Pi Calculus: Mobile Values, New Names, and Secure Communication. *Journal of the ACM (JACM)* (2018).

- [2] Lukas Aumayr, Oguzhan Ersoy, Andreas Erwig, Sebastian Faust, Kristina Hostáková, Matteo Maffei, Pedro Moreno-Sanchez, and Siavash Riahi. 2021. Generalized channels from limited blockchain scripts and adaptor signatures. In *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 635–664.
- [3] Lukas Aumayr, Matteo Maffei, Oguzhan Ersoy, Andreas Erwig, Sebastian Faust, Siavash Riahi, Kristina Hostáková, and Pedro Moreno-Sanchez. 2021. Bitcoin-compatible virtual channels. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 901–918.
- [4] Michael Backes, Jannik Dreier, Steve Kremer, and Robert Künnemann. 2017. A novel approach for reasoning about liveness in cryptographic protocols and its application to fair exchange. In *2017 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 76–91.
- [5] Gilles Barthe, Ugo Dal Lago, Giulio Malavolta, and Itsaka Rakotonirina. 2022. Tidy: Symbolic Verification of Timed Cryptographic Protocols. In *ACM Conference on Computer and Communications Security (CCS)*.
- [6] David Basin, Cas Cremers, Jannik Dreier, Simon Meier, Ralf Sasse, and Benedikt Schmidt. 2019. *Tamarin prover manual*. <https://tamarin-prover.github.io/>.
- [7] David Basin, Ralf Sasse, and Jorge Toro-Pozo. 2021. The EMV standard: Break, fix, verify. In *S&P*.
- [8] Karthikeyan Bhargavan, Vincent Cheval, and Christopher Wood. 2022. A symbolic analysis of privacy for tls 1.3 with encrypted client hello. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*.
- [9] Karthikeyan Bhargavan, Charlie Jacomme, Franziskus Kiefer, and Rolf Schmidt. 2024. Formal verification of the {PQXDH} {Post-Quantum} key agreement protocol for end-to-end secure messaging. In *USENIX*.
- [10] Bruno Blanchet, Ben Smyth, Vincent Cheval, and Marc Sylvestre. 2020. *Automatic Cryptographic Protocol Verifier, User Manual and Tutorial*. <https://prosecco.gforge.inria.fr/personal/bblanche/proverif/manual.pdf>.
- [11] Dan Boneh, Joseph Bonneau, Benedikt Bünz, and Ben Fisch. 2018. Verifiable delay functions. In *Annual international cryptology conference (CRYPTO)*. Springer, Santa Barbara, CA, USA, 757–788.
- [12] Dan Boneh and Moni Naor. 2000. Timed commitments. In *Annual international cryptology conference (CRYPTO)*. Springer, Santa Barbara, CA, USA, 236–254.
- [13] Joseph Bonneau, Benedikt Bünz, Miranda Christ, and Yuval Efron. 2025. Good things come to those who wait: Dishonest-majority coin-flipping requires delay functions. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 225–253.
- [14] Stefan Brands and David Chaum. 1993. Distance-bounding protocols. In *Workshop on the Theory and Application of Cryptographic Techniques*. Springer.
- [15] Sergiu Bursuc and Steve Kremer. 2019. Contingent payments on a public ledger: models and reductions for automated verification. In *European Symposium on Research in Computer Security (ESORICS)*. Springer, Luxembourg, 361–382.
- [16] Ran Canetti. 2001. Universally Composable Security: A New Paradigm for Cryptographic Protocols. In *FOCS*. IEEE, 136–145.
- [17] Stefanos Chaliasos, Denis Firsov, and Benjamin Livshits. 2024. Towards a formal foundation for blockchain rollups. *arXiv preprint arXiv:2406.16219* (2024).
- [18] Vincent Cheval, Charlie Jacomme, Steve Kremer, and Robert Künnemann. 2022. Saptic+ : protocol verifiers of the world, unite! In *USENIX Security Symposium*.
- [19] Hao Chung, Elisaweta Masserova, Elaine Shi, and Sri AravindaKrishnan Thyagarajan. 2026. Rapidash: Atomic Swaps Secure Under User-Miner Collusion. In *Financial Cryptography and Data Security*, Christina Garman and Pedro Moreno-Sanchez (Eds.). Springer Nature Switzerland, Cham, 249–266.
- [20] Richard Cleve. 1986. Limits on the security of coin flips when half the processors are faulty. In *ACM symposium on Theory of computing (STOC)*.
- [21] Cas Cremers, Marko Horvat, Jonathan Hoyland, Sam Scott, and Thyla van der Merwe. 2017. A Comprehensive Symbolic Analysis of TLS 1.3. In *ACMConference on Computer and Communications Security (CCS)*.
- [22] Alexandre Debant and Stéphanie Delaune. 2019. Symbolic verification of distance bounding protocols. In *International Conference on Principles of Security and Trust (POST)*.
- [23] Vincent Diemunsch, Lucca Hirschi, and Steve Kremer. 2025. A Comprehensive Formal Security Analysis of OPC UA. In *USENIX*. <https://www.usenix.org/conference/usenixsecurity25/presentation/diemunsch>
- [24] Danny Dolev and Andrew Yao. 1983. On the security of public key protocols. *IEEE Transactions on information theory* 29, 2 (1983), 198–208.
- [25] Noemi Glaeser, Matteo Maffei, Giulio Malavolta, Pedro Moreno-Sanchez, Erkan Tairi, and Sri Aravinda Krishnan Thyagarajan. 2022. Foundations of coin mixing services. In *Proceedings of the 2022 ACM SIGSAC conference on computer and communications security*. 1259–1273.
- [26] Dennis Jackson, Cas Cremers, Katriel Cohn-Gordon, and Ralf Sasse. 2019. Seems Legit: Automated Analysis of Subtle Attacks on Protocols That Use Signatures. In *Conference on Computer and Communications Security (CCS)*. ACM. <http://doi.acm.org/10.1145/3319535.3339813>
- [27] Steve Kremer and Robert Künnemann. 2016. Automated analysis of security protocols with global state. *Journal of Computer Security (JCS)* (2016).
- [28] Arjen K Lenstra and Benjamin Wesolowski. 2015. A random zoo: sloth, unicorn, and trx. *IACR Cryptol. ePrint Arch.*.
- [29] Li Li, Jun Sun, Yang Liu, and Jin Song Dong. 2015. Verifying parameterized timed security protocols. In *International Symposium on Formal Methods (FM)*. Springer, Oslo, Norway, 342–359.
- [30] Felix Linker, Ralf Sasse, and David Basin. 2025. A Formal Analysis of Apple's {iMessage} {PQ3} Protocol. In *USENIX*.
- [31] Robin Linus, Lukas Aumayr, Zeta Avarikioti, Matteo Maffei, Andrea Pelosi, Orfeas Thyfronitis Litos, Christos Stefo, David Tse, and Alexei Zamyatin. 2025. Bridging Bitcoin to second layers via BitVM2. *Cryptology ePrint Archive* (2025).
- [32] Léonard Lys, Arthur Micoulet, and Maria Potop-Butucaru. 2021. R-SWAP: Relay Based Atomic Cross-Chain Swap Protocol. In *Algorithmic Aspects of Cloud Computing*, Gianlorenzo D'Angelo and Othon Michail (Eds.). Springer International Publishing, Cham, 18–37.
- [33] Sjouke Mauw, Zach Smith, Jorge Toro-Pozo, and Rolando Trujillo-Rasua. 2018. Distance-bounding protocols: Verification without time and location. In *IEEE Symposium on Security and Privacy (S&P)*. IEEE, San Francisco, USA, 549–566.
- [34] Simon Meier. 2013. *Advancing Automated Security Protocol Verification*. Ph. D. Dissertation. ETH Zurich.
- [35] Mohsen Minaei, Duc V Le, and Pedro Moreno-Sanchez. 2026. OptiBridge: A Trustless, Cost-Efficient Bridge Between the Lightning Network and Ethereum. *Cryptology ePrint Archive* (2026).
- [36] Peifang Ni, Anqi Tian, and Jing Xu. 2025. Warning! The Timeout T Cannot Protect You From Losing Coins: PipeSwap: Forcing the Timely Release of a Secret for Atomic Cross-Chain Swaps. In *Symposium on Security and Privacy, SP*. IEEE, 1566–1583.
- [37] Itsaka Rakotonirina, Gilles Barthe, and Clara Schneidewind. 2024. Decision and complexity of Dolev-Yao hyperproperties. In *ACM SIGPLAN Symposium on Principles of Programming Languages (POPL)*.
- [38] Ronald L Rivest, Adi Shamir, and David A Wagner. 1996. Time-lock puzzles and timed-release crypto. Massachusetts Institute of Technology. Laboratory for Computer Science.
- [39] Giulia Scaffino, Lukas Aumayr, Mahsa Bastankhah, Zeta Avarikioti, and Matteo Maffei. 2024. Alba: The dawn of scalable bridges for blockchains. *Cryptology ePrint Archive* (2024).
- [40] Sri AravindaKrishnan Thyagarajan, Giulio Malavolta, and Pedro Moreno-Sanchez. 2022. Universal atomic swaps: Secure exchange of coins across all blockchains. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1299–1316.
- [41] Sri Aravinda Krishnan Thyagarajan, Adithya Bhat, Giulio Malavolta, Nico Dötting, Aniket Kate, and Dominique Schröder. 2020. Verifiable Timed Signatures Made Practical. *ACM Conference on Computer and Communications Security (CCS)*.

A Open Science

We release an artifact including our tool implementation, models of the protocols analysed in the paper, and the scripts to run the experiments. Available at <https://gitlab.com/irakoton/tidy-automation>.

B Ethical Considerations

We develop an analyser for cryptographic protocols achieving security built on time delays. Our tool verifies the fairness of a set of protocols relying on timed cryptography including, e.g., verifiable delay functions or sealed-bid auctions. This benefits the research community by verifying that these protocols achieve fairness.

We also analyse an atomic swap [40], unveiling two attacks on its setup phase. The primarily affected stakeholders are the authors of [40] and future implementers of this protocol. As the protocol in [40] is a theoretical design and we found no evidence of real-world implementations of it, we do not see immediate risk to end-user funds at existing blockchains. In accordance with responsible disclosure practices, we notified the authors of [40] about our findings to allow for the refinement of their design.

While, similar to other existing tools like Tamarin or ProVerif, the tool developed in this work could theoretically be used to identify vulnerabilities in other timed-cryptographic protocols, we believe the benefit to the research community (i.e., enabling the design of provably secure protocols) outweighs the risk of misuse of the tool. By automating the verification of security properties, our tool empowers designers to fix flaws before deployment.